

Salman Abdul Moiz

XAI-BASED STACKED ATTENTION DEEP LEARNING FRAMEWORK FOR IOT BOTNET ATTACK DETECTION USING L...

 CheckingsimalrityofReports

 July_Dec2026

 University of Hyderabad

Document Details

Submission ID

trn:oid::1:3613442726

Submission Date

Jul 19, 2026, 10:37 AM GMT+5:30

Download Date

Jul 19, 2026, 11:00 AM GMT+5:30

File Name

MTech_Thesis_Main_Content.pdf

File Size

2.2 MB

93 Pages

23,184 Words

144,299 Characters

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

-  **160 Not Cited or Quoted 7%**
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5%  Internet sources
- 6%  Publications
- 3%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 160** Not Cited or Quoted 7%
Matches with neither in-text citation nor quotation marks
- 1** Missing Quotations 0%
Matches that are still very similar to source material
- 0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 5% Internet sources
- 6% Publications
- 3% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1

Publication

Paolo Ferro, Harinadh Vemanaboina, Chander Prakash. "Computational Techniqu... <1%
- 2

Publication

Bal S. Virdee, Tanweer Ali, Jaume Anguera, Suman Lata Tripathy. "Connecting Int... <1%
- 3

Student papers

University of Northumbria at Newcastle <1%
- 4

Student papers

Universiti Kuala Lumpur <1%
- 5

Publication

Wang Wu, Harrou Fouzi, Bouyeddo Benamar, Senouci Sidi-Mohammed, Sun Ying. ... <1%
- 6

Internet

ses.library.usyd.edu.au <1%
- 7

Publication

Bhabani Prasanna Pattanaik, Dharendra Kumar Shukla, Sambit Satpathy, Kamala... <1%
- 8

Student papers

University of Strathclyde <1%
- 9

Internet

www.mdpi.com <1%
- 10

Publication

Dinesh Goyal, Payal Bansal, Ritam Dutta, Madhav Sharma. "Intelligent Systems U... <1%

11	Student papers	University of Westminster	<1%
12	Publication	Wasswa, Hassan. "A Deep Learning-Based Approach for Detection and Tracking o...	<1%
13	Internet	etd.aau.edu.et	<1%
14	Internet	mdpi-res.com	<1%
15	Internet	computing.fud.edu.ng	<1%
16	Internet	arxiv.org	<1%
17	Internet	rest.neptune-prod.its.unimelb.edu.au	<1%
18	Publication	Naga Leela Gangala, Sireesha P, Samyu J N, Vyshnavi G, Shabhareesh A. "Towar...	<1%
19	Internet	www.irjms.com	<1%
20	Publication	"Machine Learning Technologies on Energy Economics and Finance", Springer Sci...	<1%
21	Internet	afriresearch.com	<1%
22	Internet	jdmdc.com	<1%
23	Internet	jutif.if.unsoed.ac.id	<1%
24	Student papers	CSU, San Jose State University	<1%

25	Publication	Hesham Kamal, Maggie Mashaly. "Robust Intrusion Detection System Using an I...	<1%
26	Internet	gjeta.com	<1%
27	Publication	Ahsan Nazir, Jingsha He, Nafei Zhu, Ahsan Wajahat, Xiangjun Ma, Faheem Ullah, S...	<1%
28	Publication	M Arunika, S Saranya, S Charulekha, S Kabilarajan, G Kesavan. "A Survey on Explai...	<1%
29	Student papers	Universiti Teknologi MARA	<1%
30	Publication	"Machine Learning for Cyber Physical System: Advances and Challenges", Springe...	<1%
31	Publication	Saiyed, Makhdumabanu Farukali. "Adaptive Systems for Ddos Attacks Detection a...	<1%
32	Publication	Sukhpreet Kaur, Amanpreet Kaur, Manish Kumar. "Recent Advances in Computati...	<1%
33	Internet	jett.labosfor.com	<1%
34	Internet	cybersecurity.springeropen.com	<1%
35	Student papers	University of Greenwich	<1%
36	Publication	V. Sekhar, P. Ajay Kumar Reddy, K. Logesh, T. Raghunadha Reddy, Dara Raju. "Ne...	<1%
37	Internet	www.coursehero.com	<1%
38	Internet	www.journal-isi.org	<1%

39	Student papers	Liverpool John Moores University	<1%
40	Internet	norma.ncirl.ie	<1%
41	Internet	takeoffprojects.com	<1%
42	Internet	www.nerdpublication.com	<1%
43	Student papers	Bocconi University	<1%
44	Internet	github.com	<1%
45	Student papers	BITS, Pilani-Dubai	<1%
46	Student papers	University of Mines and Technology	<1%
47	Internet	download.bibis.ir	<1%
48	Internet	dspace.bracu.ac.bd:8080	<1%
49	Internet	dspace.lib.ntua.gr	<1%
50	Internet	ebin.pub	<1%
51	Internet	isjem.com	<1%
52	Internet	journal.ijprse.com	<1%

53	Internet	unsworks.unsw.edu.au	<1%
54	Internet	www.irjweb.com	<1%
55	Internet	www.warse.org	<1%
56	Student papers	Gomal University	<1%
57	Student papers	Informatics Education Limited	<1%
58	Student papers	Malta College of Arts,Science and Technology	<1%
59	Student papers	Monash University	<1%
60	Publication	R. K. Naresh, R. K. Gupta, Rajbir Garg, R. K. Sohane, J. P. Singh, O. P. Gupta, Arun K...	<1%
61	Internet	aislyn.in	<1%
62	Internet	cris.brighton.ac.uk	<1%
63	Internet	rosap.ntl.bts.gov	<1%
64	Publication	Shankar Babu, Mahesh Babu Kota. "Synergies in Smart and Virtual Systems using...	<1%
65	Student papers	Universiti Malaysia Perlis	<1%
66	Student papers	University of Huddersfield	<1%

67	Internet	eprints.utm.my	<1%
68	Internet	eprints.whiterose.ac.uk	<1%
69	Internet	www.ijraset.com	<1%
70	Internet	www.jait.us	<1%

ABSTRACT

The rapid proliferation of Internet of Things (IoT) devices across smart homes, healthcare systems, industrial automation, and critical infrastructure has significantly expanded the attack surface available to malicious actors, with botnet attacks emerging as one of the most persistent and damaging threats to these interconnected environments. Owing to their limited computational capacity and often minimal built-in security mechanisms, IoT devices are particularly susceptible to compromise, enabling attackers to assemble large networks of infected devices capable of launching Distributed Denial-of-Service attacks, data exfiltration, and unauthorized system access. Traditional signature-based and rule-driven intrusion detection systems have proven increasingly inadequate against such threats, as their reliance on predefined attack patterns leaves them unable to reliably detect novel, evolving, or obfuscated botnet behavior within the high-volume, heterogeneous traffic characteristic of IoT networks.

While deep learning-based detection systems have demonstrated considerable improvement in identifying complex attack patterns within IoT traffic, the majority of existing approaches operate as opaque, black-box classifiers, offering no insight into the reasoning underlying their predictions. This lack of transparency presents a significant obstacle to their adoption in operational cybersecurity settings, where analysts must be able to interpret, verify, and justify automated detection decisions before acting upon them. This absence of interpretability, combined with the observation that existing hybrid deep learning architectures rarely incorporate mechanisms capable of selectively emphasising the most relevant traffic features, motivated the development of the present work.

This thesis presents an XAI-Based Stacked Attention Deep Learning Framework designed to address these limitations by combining accurate, multi-class botnet attack detection with transparent, human-interpretable explanations of model behavior. The specific objectives of the proposed work were to design a stacked deep learning architecture capable of classifying IoT network traffic into normal and multiple botnet attack categories, to

enhance this architecture through the integration of an attention mechanism capable of prioritising the most informative traffic features, and to incorporate Explainable Artificial Intelligence techniques capable of clarifying the basis for individual predictions. The proposed framework was developed and evaluated using the UNSW-NB15 benchmark dataset, a widely adopted intrusion detection dataset comprising labelled network traffic spanning normal activity and multiple categories of malicious behavior, including Denial-of-Service, Exploits, Reconnaissance, Backdoors, Shellcode, Worms, Generic, and Analysis attacks.

Experimental results demonstrated that the proposed stacked attention architecture achieved the strongest overall classification performance among the models evaluated, consistently outperforming the individual ANN, CNN, and LSTM baselines across the accuracy, precision, recall, and F1-score metrics considered, indicating that the combination of attention-based feature weighting with convolutional and recurrent processing offers a measurable advantage over any single architectural approach applied independently. To address the interpretability limitations inherent in the trained model, three complementary Explainable AI techniques, namely LIME, SHAP, and Integrated Gradients, were integrated into the framework, each offering a distinct perspective on feature-level prediction attribution: LIME through local surrogate modelling, SHAP through game-theoretic Shapley value estimation, and Integrated Gradients through gradient-based attribution along an interpolated input path.

To demonstrate the practical applicability of the proposed framework, the trained model and its associated explainability modules were deployed within a Flask-based web application, incorporating user registration, authentication, dataset upload, and result visualization functionality, allowing authorized users to submit traffic data and obtain both classification outcomes and corresponding explanations through an accessible interface. The findings of this study confirm that combining architectural sophistication with integrated explainability is both technically feasible and practically valuable, offering a detection framework that is not only accurate but also transparent and trustworthy for real-world security applications.

LIST OF CONTENTS

S.NO	CONTENT	PAGE NO
1	INTRODUCTION	1
	1.1 Introduction	
	1.2 Background of IoT and Botnet Attacks	
	1.3 Problem Statement	
	1.4 Aim of the Proposed Work	
	1.5 Objectives	
	1.6 Motivation	
	1.7 Scope of the Work	
	1.8 Organization of the Thesis	
2	LITERATURE SURVEY	7
	2.1 Introduction to Literature Survey	
	2.2 Foundational Hybrid and Ensemble Deep Learning Approaches	
	2.3 Deep Learning Architectures for IoT Botnet Detection	
	2.4 Benchmark Datasets and Performance Evaluation	
	2.5 Explainable Artificial Intelligence in Cybersecurity	
	2.6 Adversarial Robustness and Emerging Challenges	
	2.7 Comparative Summary of Reviewed Literature	
	2.8 Summary of Literature Survey	
	2.9 Research Gap	

3	SYSTEM DESIGN	26
	3.1 Importance	
	3.2 UML Diagrams	
	3.3 Functional Requirement	
4	IMPLEMENTATION	35
	4.1 Overall Implementation Overview	
	4.2 Dataset Loading and Preparation	
	4.3 Data Preprocessing	
	4.4 ANN Implementation	
	4.5 CNN Implementation	
	4.6 LSTM Implementation	
	4.7 Stacked Attention Model Implementation	
	4.8 Explainable AI (XAI) Implementation	
	4.9 Flask Web Application Implementation	
	4.10 Prediction Workflow	
	4.11 Module Interaction	
	4.12 Chapter Summary	
5	TESTING	53
	5.1 Testing Overview	
	5.2 Testing Environment	
	5.3 Test Cases	
	5.4 Functional Testing	
	5.5 Integration Testing	
	5.6 Performance Testing	

	5.7 User Acceptance Testing	
	5.8 Security Testing	
	5.9 Testing Summary	
6	RESULTS	67
	Experimental Results	
	Model Performance Analysis	
	Explainability Results	
7	CONCLUSION	83
	7.1 Conclusion	
	7.2 Future Work	
8	REFERENCES	88

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
2.1	Comparative Summary of Reviewed Literature	22
5.1	Test Case TC-01: User Registration	57
5.2	Test Case TC-02: User Login	57
5.3	Test Case TC-03: Dataset Upload	58
5.4	Test Case TC-04: Data Validation	58
5.5	Test Case TC-05: Model Loading	59
5.6	Test Case TC-06: Botnet Attack Prediction	59
5.7	Test Case TC-07: Explainable AI Visualization	59
5.8	Test Case TC-08: Result Display	60
5.9	Test Case TC-09: Invalid Input Handling	60

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
2.1	Proposed System Architecture	25
3.1	Use Case Diagram	28
3.2	Class Diagram	28
3.3	Sequence Diagram	29
3.4	Activity Diagram	30
3.5	System Architecture	32
3.6	Deployment Diagram	33
3.7	Component Diagram	34
4.1	UNSW-NB15 Dataset	40
6.1	LIME Explanation	68
6.2	SHAP Explanation	69
6.3	Integrated Gradient Explanation	70
6.4	Home Page of the Application	71
6.5	User Login Interface	71
6.6	User Registration Interface	72
6.7	Dataset Upload Interface	72
6.8	Dataset Preprocessing Output	73
6.9	Detected Attack Results	73
6.10	Attack Class Distribution in the Dataset	74
6.11	ANN Model Training and Validation Performance	74
6.12	CNN Model Training and Validation Performance	75
6.13	RNN Model Training and Validation Performance	75
6.14	Training Performance of the Proposed Stacked Deep Learning Model	76
6.15	Training Performance of the Proposed Stacked Attention Model	76
6.16	Performance Comparison of Deep Learning Models	77
6.17	LIME-Based Explanation of Model Prediction	77
6.18	SHAP Summary Plot for Feature Importance	78
6.19	Integrated Gradients Feature Attribution	78

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 Introduction

The Internet of Things (IoT) has fundamentally transformed the manner in which everyday devices communicate, automate, and exchange information, enabling billions of interconnected devices to operate across domains such as smart homes, healthcare monitoring, industrial automation, and urban infrastructure management. This proliferation of connected devices has delivered substantial gains in efficiency, convenience, and automation, yet it has simultaneously introduced a considerably expanded attack surface for cybercriminals to exploit. Because IoT devices are frequently designed with cost and power efficiency prioritised over robust security, they often lack the computational capacity to support conventional security mechanisms such as strong encryption, intrusion prevention agents, or frequent firmware patching. This combination of scale and inherent vulnerability has made IoT ecosystems an increasingly attractive target for botnet attacks, in which large numbers of compromised devices are assembled into coordinated networks capable of executing large-scale malicious operations.

Detecting such attacks reliably, while also ensuring that detection decisions can be understood and trusted by the security personnel responsible for acting on them, forms the central technical challenge addressed by this thesis. This chapter introduces the background of IoT botnet threats, defines the problem addressed by the proposed work, and outlines the aim, objectives, motivation, and scope of the research undertaken, concluding with an overview of how the remainder of this document is organised.

1.2 Background of IoT and Botnet Attacks

A botnet refers to a network of compromised computing devices, commonly referred to as bots, that are placed under the remote control of an attacker, often termed a botmaster, without the knowledge or consent of their legitimate owners. Once assembled, a botnet can be directed to carry out a range of coordinated malicious activities, including Distributed Denial-of-Service (DDoS) attacks capable of overwhelming targeted services, large-scale data theft, unauthorised access to connected systems, and the further propagation of malware to additional vulnerable devices. Within the IoT context, the scale of this threat is magnified considerably by the sheer

number of connected endpoints and by the tendency of many IoT devices to remain in continuous, unattended operation, offering attackers a persistent and largely unmonitored foothold from which to launch further activity.

The structural characteristics of modern botnets have also grown increasingly sophisticated, with attackers employing peer-to-peer communication topologies and encrypted or disguised command-and-control channels specifically to evade detection by conventional security monitoring. Traditional intrusion detection approaches, which rely predominantly on signature matching or predefined rule sets derived from previously observed attack patterns, are consequently ill-suited to identifying new or evolving botnet variants, particularly zero-day threats for which no prior signature exists. These limitations have driven growing interest in machine learning and deep learning-based detection approaches, which are capable of learning discriminative patterns directly from network traffic data rather than depending on manually defined rules. Deep learning architectures, including Convolutional Neural Networks, Recurrent Neural Networks, and Long Short-Term Memory networks, have shown particular promise in this regard owing to their capacity to automatically extract complex spatial and temporal patterns from high-dimensional traffic data. However, as these models grow more capable, they simultaneously become more opaque, and the resulting lack of transparency has emerged as a significant barrier to their confident adoption within operational cybersecurity environments, a concern that lies at the heart of the work presented in this thesis.

1.3 Problem Statement

IoT network environments generate high-volume, high-velocity traffic that must be analysed continuously to identify malicious activity, yet existing detection approaches face two interconnected shortcomings that limit their practical effectiveness. First, traditional rule-based and signature-based intrusion detection systems are fundamentally reactive in nature, capable of recognising only attack patterns that have been explicitly defined in advance, and are therefore poorly equipped to detect the evolving, obfuscated, and zero-day attack behaviours characteristic of modern botnets. Second, while deep learning-based detection systems address this shortcoming by learning directly from traffic data and achieving substantially improved detection accuracy, they typically function as black-box models whose internal decision-making processes remain inaccessible to the security analysts who must interpret and act upon their outputs. A detection system that produces accurate classifications without any accompanying justification offers limited practical value in a security operations context,

where decisions to isolate a device, block traffic, or escalate an incident require a defensible rationale rather than an unexplained label.

The problem addressed by this thesis is therefore twofold: the need for a detection architecture capable of accurately classifying diverse categories of botnet attack traffic within IoT environments, and the need to embed genuine interpretability into that architecture, such that its predictions can be understood, verified, and trusted by the analysts responsible for responding to them.

1.4 Aim of the Proposed Work

The aim of this project is to design and implement an XAI-Based Stacked Attention Deep Learning Framework for accurate and interpretable botnet attack detection in IoT network environments. The proposed framework combines a stacked deep learning architecture, enhanced with an attention mechanism, for the classification of network traffic, together with Explainable Artificial Intelligence techniques, namely LIME, SHAP, and Integrated Gradients, that provide transparent, feature-level justification for the model's predictions.

1.5 Objectives

The specific objectives of this project are as follows:

1. To design and implement a stacked deep learning architecture that combines convolutional feature extraction, an attention mechanism, and recurrent sequence modelling for the classification of IoT network traffic.
2. To accurately distinguish between normal network activity and multiple categories of botnet-related attack traffic using the UNSW-NB15 benchmark dataset.
3. To evaluate the contribution of the attention mechanism by comparing the performance of the proposed stacked attention model against individually implemented ANN, CNN, and LSTM baseline architectures.
4. To integrate LIME, SHAP, and Integrated Gradients into the detection pipeline, enabling the specific traffic features responsible for a given prediction to be identified and interpreted.
5. To evaluate the overall effectiveness of the proposed framework using standard classification metrics, namely Accuracy, Precision, Recall, and F1-Score.
6. To deploy the trained model and its explainability modules within a functional, authenticated web application.

1.6 Motivation

The motivation for this project arises from the widening gap between the detection accuracy achieved by modern deep learning-based cybersecurity systems and the practical trust these systems are able to command among the analysts who rely on them. As IoT networks continue to expand in scale and complexity, the volume of traffic requiring continuous security monitoring far exceeds what manual analysis can reasonably accommodate, making automated, learning-based detection an operational necessity rather than a convenience. At the same time, the black-box nature of most existing deep learning detection systems means that their outputs are frequently accepted or dismissed without genuine understanding, undermining the analyst's ability to distinguish a correctly identified threat from a false alarm, or to explain a detection decision to stakeholders who require justification for security actions taken.

This tension between predictive performance and interpretability motivated the pursuit of a framework in which these two objectives are treated as equally important design goals rather than competing priorities. The incorporation of an attention mechanism was similarly motivated by the observation that not all traffic features contribute equally to a given classification decision, and that a model capable of learning to prioritise the most relevant features may offer benefits both in terms of classification performance and in terms of producing more focused, meaningful explanations. Ultimately, this project is motivated by the broader goal of contributing toward cybersecurity systems that are not only accurate, but also transparent, accountable, and genuinely usable within real-world security operations.

1.7 Scope of the Work

The scope of this study is centred on the design, implementation, and evaluation of an XAI-Based Stacked Attention Deep Learning Framework for the detection of botnet attacks within IoT network traffic. The framework is developed and evaluated using the UNSW-NB15 dataset, encompassing normal traffic alongside multiple categories of attack behaviour, including Denial-of-Service, Exploits, Fuzzers, Reconnaissance, Backdoors, Shellcode, Worms, Generic, and Analysis attacks. The work encompasses the complete pipeline required to achieve this objective, including dataset preprocessing, the design and training of individual ANN, CNN, and LSTM baseline models, the design and training of the proposed stacked attention architecture, the integration of LIME, SHAP, and Integrated Gradients for prediction explanation, and the development of a Flask-based web application through which the complete framework can be accessed and evaluated by an authenticated user.

The scope of this work does not extend to real-time traffic capture from live network infrastructure, deployment across distributed or federated IoT environments, or hardening of the accompanying web application for production-grade security, as these aspects fall outside the boundaries of the present study and are identified instead as directions for future extension in the concluding chapter of this thesis. The evaluation presented in this work is similarly confined to the UNSW-NB15 dataset and does not extend to validation against additional benchmark datasets such as Bot-IoT, though the framework's underlying methodology is designed to be adaptable to such extensions.

1.8 Organization of the Thesis

The remainder of this thesis is organised as follows. Chapter 2 presents a review of existing literature relevant to IoT botnet detection, hybrid and individual deep learning architectures, benchmark dataset development, and the application of Explainable AI within cybersecurity, concluding with an identification of the specific research gaps addressed by this work. Chapter 3 describes the system design of the proposed framework, including its architectural workflow, UML-based modelling of system components, and the hardware and software requirements underlying its implementation. Chapter 4 details the implementation of the proposed framework, covering dataset preparation, data preprocessing, the individual ANN, CNN, and LSTM models, the proposed stacked attention architecture, the integration of the LIME, SHAP, and Integrated Gradients explainability modules, and the development of the accompanying Flask web application. Chapter 5 presents the testing methodology and results, encompassing functional, integration, performance, user acceptance, and security testing conducted across the implemented system. Chapter 6 discusses the experimental results obtained from the trained models, including comparative performance evaluation and the explainability outputs generated for selected predictions. Chapter 7 concludes the thesis with a summary of the work undertaken and an outline of promising directions for future research.

LITERATURE SURVEY

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction to Literature Survey

A structured review of existing research forms an essential precursor to the design of any new detection framework, as it enables the identification of methodological gaps, recurring limitations, and unresolved challenges that a proposed system may seek to address. In the domain of IoT botnet attack detection, the body of research spans several interconnected areas, including traditional machine learning-based intrusion detection, deep learning architectures applied to network traffic classification, benchmark dataset development, and, more recently, the incorporation of Explainable Artificial Intelligence to address the interpretability limitations inherent in complex detection models. Reviewing this literature not only situates the present work within the broader research landscape but also clarifies the specific technical contribution that the proposed XAI-Based Stacked Attention Deep Learning Framework seeks to make.

The volume of research directed toward IoT security has grown substantially in recent years, driven largely by the proliferation of connected devices and the corresponding expansion of the attack surface available to malicious actors. Within this growing body of work, several recurring patterns can be observed. A significant proportion of studies concentrate on improving raw classification accuracy through increasingly elaborate combinations of neural network architectures, often demonstrating strong empirical results on benchmark datasets such as Bot-IoT and UNSW-NB15. A comparatively smaller, though steadily increasing, subset of studies addresses the transparency of these models, recognising that high accuracy alone is insufficient for practical deployment in security-sensitive environments where analysts must be able to justify automated decisions. The following sections examine this literature across four thematic groupings, beginning in this section with foundational hybrid and ensemble deep learning approaches, before proceeding in subsequent sections to architectures specifically applied to IoT and network traffic contexts, dataset and benchmarking contributions, and the emerging intersection between deep learning and explainability. This thematic organisation is intended to allow the relationships between related studies to be

examined more clearly than would be possible through a purely chronological or alphabetical treatment of the literature.

2.2 Foundational Hybrid and Ensemble Deep Learning Approaches

Hybrid and ensemble deep learning architectures have emerged as a dominant direction in recent botnet detection research, motivated by the observation that individual neural network types tend to specialise in capturing particular aspects of network traffic behaviour, whether spatial, temporal, or sequential, and that combining them can yield representational advantages unattainable by any single architecture in isolation. The four studies reviewed in this section each pursue this hybridisation strategy, though they differ meaningfully in the specific combination of architectures employed, the datasets used for validation, and the extent to which interpretability was considered as part of the design.

Ali et al. [1] proposed a hybrid deep learning model, referred to as ACLR, that integrates Artificial Neural Networks, Convolutional Neural Networks, Long Short-Term Memory networks, and Recurrent Neural Networks within a single detection pipeline for IoT botnet traffic. The underlying objective of this work was to overcome the accuracy ceiling observed in single-architecture detection systems by allowing each constituent model to contribute complementary feature representations, with CNN layers responsible for spatial feature extraction, LSTM and RNN components capturing temporal and sequential dependencies, and the ANN component serving as the final integrating and classification layer. The model was evaluated on IoT network traffic data and reported an accuracy exceeding ninety-six percent, alongside a strong ROC-AUC score, indicating that the ensemble approach substantially outperformed the individual architectures from which it was constructed. The principal strength of this study lies in its systematic demonstration that architectural diversity within a stacked pipeline translates into measurable performance gains, and its reported results directly informed the architectural direction adopted in the present project, particularly the decision to combine convolutional and recurrent components. However, the study does not incorporate any mechanism for explaining its predictions, leaving the reasoning behind individual classifications entirely opaque to a downstream analyst, a limitation that is directly addressed by the attention and explainability components introduced in the present work. Given that ACLR [1] represents the closest architectural precedent to the stacked model developed in this

project, its findings are treated as a primary point of comparison throughout the remainder of this thesis rather than as a peripheral reference.

Alzahrani and Bamhdi [21] examined a comparable hybridisation strategy, developing a combined deep learning model intended specifically for detecting botnet attacks across IoT network environments. Rather than treating the constituent networks as independent classifiers evaluated separately, the authors constructed a unified pipeline in which multiple deep learning components jointly contribute to the final classification decision, with the explicit aim of improving robustness against the diverse and evolving attack patterns characteristic of IoT traffic. Their evaluation demonstrated that the hybrid configuration achieved higher classification accuracy than conventional single-model baselines drawn from earlier literature, reinforcing the broader finding that architectural combination offers tangible benefits over standalone deep learning classifiers in this domain. Despite these performance gains, the reported model exhibited considerable computational overhead relative to simpler architectures, a consequence of the added complexity introduced by combining multiple deep learning components, and the study offers no discussion of interpretability or the practical means by which a security analyst might understand the basis for a given prediction. This combination of high computational cost and an absence of explanatory capability underscores the trade-off that much of the existing hybrid-model literature has yet to resolve, and which the present work attempts to address through the more selectively weighted attention mechanism described in Chapter 4, rather than through the indiscriminate concatenation of multiple full-scale sub-networks.

Popoola et al. [24], in a separate contribution addressing deep learning-based intrusion detection for IoT environments, focused on refining the feature learning process itself rather than concatenating multiple independent architectures. Their framework was designed to extract informative representations directly from IoT network traffic and to classify malicious activity with an emphasis on strong generalisation performance across varied attack types. Experimental evaluation confirmed that the proposed approach was capable of accurately distinguishing between benign and malicious traffic across several categories of cyberattack, demonstrating the general suitability of deep feature learning for IoT-specific intrusion detection tasks. Nevertheless, the study concentrated almost exclusively on maximising classification metrics, and the authors provided no discussion of how or why the model arrived at particular decisions, leaving a clear interpretability gap consistent with the broader pattern

observed across this theme. This emphasis on accuracy without accompanying transparency reflects a methodological tendency that the present project deliberately departs from by treating explainability as a co-equal design objective alongside detection performance.

In a related but architecturally distinct contribution, Popoola et al. [26] proposed a stacked recurrent neural network specifically designed for botnet detection within smart home environments, a setting that shares many structural similarities with the broader IoT contexts addressed elsewhere in this literature. This work is of particular relevance to the present project because it adopts a stacking methodology, layering multiple recurrent components to progressively refine the learned representation of network traffic, rather than relying on a single recurrent layer. The stacked recurrent design was shown to improve detection performance over shallower recurrent baselines, suggesting that depth and layered refinement, rather than architectural diversity alone, can meaningfully contribute to classification accuracy in botnet detection tasks. The study, however, remains confined to recurrent processing and does not explore the incorporation of convolutional feature extraction or any attention-based weighting of learned features, nor does it address the interpretability of its stacked predictions. This narrower architectural scope, when considered alongside the broader hybrid approaches of Ali et al. [1] and Alzahrani and Bamhdi [21], helps to justify the specific architectural configuration adopted in the present work, which combines convolutional feature extraction, attention-based weighting, and recurrent sequence modelling within a single stacked pipeline, while additionally embedding explainability directly into the overall detection framework rather than treating it as an unaddressed extension.

2.3 Deep Learning Architectures for IoT Botnet Detection

While the studies discussed in the preceding section approached botnet detection primarily through the combination of multiple architectures within a single hybrid pipeline, a parallel and equally significant body of research has instead concentrated on refining individual deep learning architectures, such as convolutional networks or recurrent networks used independently, to extract meaningful patterns directly from raw or minimally processed network traffic. This theme brings together five such studies, each contributing a distinct perspective on how a specific deep learning paradigm can be adapted to the particular characteristics of botnet-generated traffic, ranging from raw packet-level analysis to convolutional feature engineering and deep classification pipelines.

Shahhosseini et al. [3] investigated the use of deep learning models operating directly on raw network traffic data, deliberately avoiding the extensive manual feature engineering that characterises many conventional intrusion detection pipelines. The central objective of their work was to determine whether a deep learning model could learn discriminative representations of botnet behaviour directly from unprocessed traffic captures, thereby reducing the dependence on domain expertise typically required to hand-craft effective traffic features. Their methodology involved feeding raw traffic sequences into a deep neural architecture capable of automatically extracting hierarchical patterns from the input stream, with model performance subsequently assessed against network traffic data containing labelled botnet activity. The study reported improved detection accuracy relative to approaches reliant on manually engineered features, demonstrating that raw traffic data, when paired with an appropriately deep architecture, can carry sufficient discriminative information for effective classification. This finding is notable because it reduces the preprocessing burden associated with traditional feature-engineering pipelines, though it simultaneously increases the model's dependence on large volumes of training data to compensate for the absence of expert-guided feature selection. A further limitation lies in the restricted explainability of the resulting model, since operating on raw traffic sequences makes it considerably more difficult to trace a given prediction back to any single, human-interpretable traffic characteristic, in contrast to models trained on well-defined, named features such as those used in the UNSW-NB15 dataset adopted in the present work. This distinction reinforces one of the key motivations behind the present study's use of a structured, feature-based dataset, as it allows the LIME, SHAP, and Integrated Gradients explanations discussed in Chapter 4 to be expressed in terms of clearly identifiable traffic attributes rather than raw packet sequences.

Homayoun et al. [4] proposed BoTShark, a deep learning-based framework specifically designed to identify botnet traffic through the analysis of network flow characteristics associated with command-and-control communication. The motivation underlying this work stemmed from the recognition that many botnet families exhibit distinctive communication patterns during their command-and-control phase, and that these patterns, if properly captured, could serve as a reliable basis for detection even when the malware payload itself varies across different botnet variants. The proposed framework employed deep learning components to automatically learn representations of these communication patterns from labelled traffic flows, with evaluation carried out against datasets containing known botnet traffic samples. The reported results indicated that BoTShark was capable of effectively distinguishing between

benign and botnet-associated flows, achieving strong detection performance across the traffic samples examined in the study. A particular strength of this approach lies in its focus on behavioural, communication-level patterns rather than payload content, making it comparatively resilient to certain evasion techniques that rely on payload obfuscation. However, the framework's effectiveness is closely tied to the specific communication patterns present in the botnet families used for training, raising concerns regarding its generalisability to botnet variants employing substantially different command-and-control strategies, and, as with several other studies in this theme, no provision is made for explaining which specific flow characteristics drove a given classification decision. The present work draws on this insight by ensuring that the stacked attention model is trained across a diverse range of attack categories present in UNSW-NB15, reducing the risk of overfitting to a narrow set of communication patterns, while its accompanying explainability modules directly address the transparency gap left unresolved by BoTShark.

Liu et al. [10] examined the application of deep learning techniques to IoT botnet detection with a particular emphasis on adapting established neural network methodologies to the resource and traffic characteristics specific to IoT environments. Their work aimed to assess how effectively deep learning models, originally developed for more general network intrusion detection tasks, could be applied to the comparatively constrained and heterogeneous traffic patterns generated by IoT devices. The methodology adopted in this study involved training a deep learning classifier on IoT-specific network traffic data and evaluating its ability to correctly distinguish between normal device behaviour and traffic associated with botnet compromise. The results demonstrated that deep learning approaches could be successfully transferred to the IoT context, achieving detection performance comparable to that observed in more conventional network settings, thereby supporting the broader premise that deep learning remains an effective tool even under the constrained and variable conditions typical of IoT traffic. Nonetheless, the study's evaluation was limited in scope with respect to the diversity of attack categories considered, and, in common with the majority of the literature reviewed thus far, it offered no mechanism for interpreting the basis of individual predictions, treating the classifier strictly as a black-box decision-making tool. The broader diversity of attack categories present in the UNSW-NB15 dataset used in the present project, spanning denial-of-service, reconnaissance, exploitation, and several additional attack types, extends beyond the narrower scope adopted by Liu et al., allowing the proposed framework to be evaluated across

a more comprehensive representation of the threats an IoT deployment might realistically encounter.

Haq and Khan [16] introduced DNNBoT, a deep neural network-based framework developed for both the detection and classification of botnet activity, distinguishing itself from purely binary detection approaches by additionally aiming to identify the specific category of botnet behaviour present within the analysed traffic. The study's objective was accordingly twofold: to reliably separate malicious from benign traffic, and to further categorise detected malicious traffic according to its underlying botnet type, a task considerably more demanding than binary classification alone. The proposed DNNBoT architecture was trained and evaluated using labelled network traffic data encompassing multiple botnet categories, with performance assessed using standard classification metrics across both the detection and categorisation tasks. The reported findings indicated that the deep neural network was capable of achieving strong classification performance across the multiple botnet categories considered, supporting the feasibility of multi-class botnet classification using deep learning. This multi-class orientation aligns closely with the classification objective pursued in the present work, which likewise seeks to distinguish among numerous distinct attack categories present in UNSW-NB15 rather than performing a simplified binary classification alone. Despite this alignment in classification scope, DNNBoT does not incorporate any architectural mechanism analogous to attention-based feature weighting, nor does it provide any interpretability layer capable of explaining why a particular traffic sample was assigned to a specific botnet category, leaving analysts without insight into the model's category-level reasoning, a gap that the explainability components of the present framework are specifically designed to address across all classified attack categories.

Chen et al. [23] proposed a botnet detection approach centred on the extraction of convolutional features from network traffic, applying neural network classification to representations derived through convolutional processing rather than relying on raw or purely statistical traffic attributes. The underlying rationale for this approach was that convolutional operations, while conventionally associated with image data, could be repurposed to extract meaningful local patterns from structured traffic representations, potentially uncovering discriminative characteristics that traditional feature-engineering approaches might overlook. Their methodology involved transforming network traffic into a format amenable to convolutional processing, followed by the application of a neural network classifier trained on the resulting

convolutional feature maps, with evaluation conducted against network traffic datasets containing labelled botnet activity. The study reported that the convolutional feature-based approach achieved effective detection performance, providing empirical support for the broader use of convolutional architectures in network traffic analysis and reinforcing a methodological direction closely related to the convolutional feature extraction stage employed in the stacked attention model developed in this project. However, the study did not extend its convolutional feature extraction approach with any subsequent attention mechanism or recurrent processing, treating the convolutional representation as the endpoint of the feature extraction process rather than as an intermediate stage feeding into further sequential refinement, and it similarly omitted any consideration of model interpretability. The present work builds directly upon the convolutional feature extraction principle validated by Chen et al., while extending it substantially through the addition of an attention mechanism and recurrent LSTM integration, as well as through the incorporation of explainability techniques entirely absent from their study.

2.4 Benchmark Datasets and Performance Evaluation

Beyond the design of detection architectures themselves, a critical strand of research within the botnet detection literature concerns the development and evaluation of benchmark datasets capable of realistically representing IoT network behaviour under both normal and adversarial conditions. Without such datasets, the comparative evaluation of detection models would lack a common empirical foundation, making it impossible to meaningfully assess whether a proposed architecture offers genuine improvement over existing approaches. This section examines two studies whose contributions centre on the construction and application of such benchmark data, providing the empirical basis upon which much of the architectural research discussed in the preceding sections, as well as the present work, ultimately depends.

Koroniotis et al. [30] introduced the Bot-IoT dataset, developed with the explicit aim of addressing the shortage of realistic, purpose-built datasets for evaluating IoT botnet detection systems. Prior to this contribution, much of the available intrusion detection research relied on datasets originally constructed for general network security purposes, which often failed to capture the specific traffic characteristics, device behaviours, and attack patterns associated with IoT environments. The methodology adopted by the authors involved simulating a realistic IoT network testbed and generating traffic encompassing both legitimate device

communication and a range of attack behaviours, including distributed denial-of-service activity, denial-of-service activity, reconnaissance, information theft, and keylogging, with each traffic flow subsequently labelled according to its associated category. The resulting dataset was made publicly available to the research community and has since become one of the most widely referenced benchmarks in IoT-focused intrusion detection research, a status reflected in its adoption as an evaluation dataset in several of the architectural studies discussed earlier in this chapter. The principal strength of this contribution lies in the breadth and realism of the attack scenarios represented, offering researchers a dataset considerably better aligned with genuine IoT threat conditions than earlier, more generic alternatives. Nevertheless, the study's contribution is confined to dataset construction and characterisation, and it does not propose or evaluate any detection methodology of its own, nor does it address any aspect of model interpretability, since the dataset itself is methodology-agnostic. In the present work, the closely related UNSW-NB15 dataset was adopted rather than Bot-IoT, owing to its broader coverage of general network intrusion categories alongside botnet-relevant attack types and its established use as a benchmark for both classification accuracy and, increasingly, explainability-focused evaluation, allowing the results obtained in this thesis to remain comparable with a wide cross-section of related literature while still reflecting the realistic, multi-category attack diversity that Koroniotis et al. demonstrated to be essential for meaningful IoT security evaluation.

Sriram et al. [12] addressed the closely related challenge of designing an effective network-flow-based approach to IoT botnet detection, with particular attention to how traffic should be represented and summarised prior to being supplied to a deep learning classifier. Rather than focusing primarily on dataset construction, this study concentrated on demonstrating how flow-level traffic representations, capturing aggregated characteristics of a communication session rather than individual packets, could be leveraged effectively within a deep learning-based detection pipeline. The methodology involved extracting flow-based features from IoT network traffic and applying a deep learning model to classify these flows as either benign or indicative of botnet activity, with the resulting performance evaluated using standard classification metrics against labelled traffic data. The findings demonstrated that flow-based representations retained sufficient discriminative information to support accurate botnet detection, reinforcing the broader methodological choice, also adopted in the present work, of relying on flow-level traffic attributes rather than raw packet captures as the primary input representation for deep learning-based classification. This flow-based orientation offers a

practical advantage in terms of computational efficiency and interpretability of individual features, since flow-level attributes such as duration, byte counts, and connection state are more readily meaningful to a human analyst than raw packet payloads. However, the study's evaluation was conducted using a comparatively narrower set of attack categories than those represented in UNSW-NB15, and, consistent with the broader pattern observed throughout this literature review, the work did not incorporate any explanatory mechanism capable of indicating which specific flow attributes most strongly influenced a given classification outcome. The present work extends this flow-based methodological foundation by applying it across the substantially broader set of attack categories available in UNSW-NB15, while directly addressing the interpretability limitation identified in Sriram et al.'s study through the integration of LIME, SHAP, and Integrated Gradients, thereby allowing the specific flow-based features responsible for each prediction to be explicitly identified and communicated to a security analyst.

2.5 Explainable Artificial Intelligence in Cybersecurity

The preceding three sections have collectively illustrated a recurring characteristic of the deep learning-based botnet detection literature: irrespective of whether a study pursues hybrid architectures, individually refined neural network designs, or carefully constructed benchmark datasets, the resulting detection models are almost universally treated as black-box systems, evaluated exclusively in terms of classification accuracy, precision, recall, and related performance metrics, with little to no attention given to how or why a given prediction was reached. This absence of interpretability carries particular significance in cybersecurity applications, where the consequences of an automated decision, such as flagging a device as compromised or classifying legitimate traffic as malicious, can directly affect operational continuity, incident response prioritisation, and analyst trust in the system. A security analyst presented with a bare classification label of "attack" or "normal" is left with no means of independently verifying the model's reasoning, distinguishing genuine detections from false positives, or identifying which traffic characteristics warrant further investigation. Explainable Artificial Intelligence has consequently emerged as a necessary complement to high-performing deep learning models in this domain, offering techniques capable of exposing the internal reasoning of otherwise opaque classifiers in a form that human analysts can interpret and act upon. This section reviews two studies that engage with this interpretability challenge from different perspectives, one surveying the broader landscape of deep learning applications

in cybersecurity while highlighting their explainability shortcomings, and the other addressing classification performance in a manner that, despite its methodological strengths, similarly leaves the interpretability question unresolved.

10 McDermott et al. [11] conducted a broad survey examining the application of deep learning techniques across the cybersecurity and intrusion detection landscape, encompassing a wide range of neural network architectures and their respective use cases in identifying malicious network activity. The objective of this survey was not to propose a new detection model but rather to consolidate and critically assess the state of deep learning-based cybersecurity research, identifying common architectural trends, recurring evaluation practices, and shared limitations across the reviewed body of work. Through this synthesis, the authors observed that deep learning models had consistently demonstrated a strong capacity for automatically extracting complex features from network traffic without requiring extensive manual feature engineering, a capability that had driven much of the field's progress in classification accuracy. However, the survey also identified a pervasive and largely unaddressed limitation across the reviewed literature: the overwhelming majority of deep learning-based intrusion detection systems examined provided no insight into the reasoning underlying their predictions, operating instead as opaque black-box classifiers whose internal decision processes remained inaccessible to the security analysts responsible for acting on their outputs. This finding is of particular relevance to the present work, as it provides direct, literature-based confirmation that the interpretability gap addressed by this thesis is not an isolated concern specific to any single study but rather a systemic characteristic of the deep learning-based intrusion detection field as a whole. The survey's breadth, however, also constitutes its principal limitation from the perspective of the present work, since it does not propose, implement, or evaluate any specific explainability technique, nor does it examine methods such as LIME, SHAP, or Integrated Gradients that could practically address the transparency gap it identifies, leaving this as an open direction that the present study directly undertakes.

25 Ahmed et al. [18] proposed a deep learning-based classification model developed specifically for the task of botnet attack detection, with the primary research objective of improving classification accuracy across multiple categories of botnet-related traffic. The methodology adopted in this study involved training a deep neural network classifier on labelled network traffic data, applying standard preprocessing and feature preparation steps before evaluating the model's ability to correctly categorise both benign and malicious traffic samples. The results

38

22 reported by the authors indicated that the proposed classification model achieved strong detection performance across the evaluated attack categories, contributing further empirical evidence to the broader literature demonstrating the effectiveness of deep learning approaches in this domain. The study's principal strength lies in its systematic evaluation of classification performance across a diverse set of botnet-related traffic categories, offering a further comparative benchmark against which the classification accuracy of the present framework can be assessed. However, in common with the majority of the classification-focused studies examined throughout this chapter, Ahmed et al. did not incorporate any interpretability mechanism into their model, and the study offers no discussion of feature-level attribution, decision transparency, or the practical means by which an analyst might understand the basis for a given classification outcome. When considered alongside the findings of McDermott et al., this study reinforces the broader pattern observed throughout the literature: that strong classification performance and model interpretability have, to date, been pursued largely as separate and independent research objectives rather than as jointly addressed design goals. It is precisely this separation that the present work seeks to overcome, by treating the integration of LIME, SHAP, and Integrated Gradients not as a supplementary addition to an otherwise conventional classifier, but as a core architectural objective pursued with the same rigour applied to the design of the stacked attention model itself, thereby positioning detection accuracy and explainability as complementary, rather than competing, priorities within a single unified framework.

2.6 Adversarial Robustness and Emerging Challenges

7 While the preceding themes have primarily examined detection accuracy and interpretability as the two dominant evaluation criteria within the botnet detection literature, a further and increasingly important dimension concerns the robustness of deep learning-based detection systems when subjected to deliberately crafted adversarial inputs. As detection models grow more sophisticated, so too does the incentive for attackers to develop traffic patterns specifically designed to evade classification, raising questions about the reliability of deep learning-based intrusion detection systems under adversarial conditions rather than under the benign evaluation settings typically reported in the literature.

Ibitoye et al. [2] examined this concern directly, analysing the vulnerability of deep learning-based intrusion detection systems for IoT networks to adversarial attacks. The research objective of this study was to assess how susceptible neural network-based classifiers are to small, deliberately crafted perturbations of input traffic features, perturbations designed to cause a misclassification without necessarily altering the underlying malicious intent of the traffic itself. The methodology employed involved training two categories of neural network classifiers, a feedforward neural network and a self-normalising neural network, on IoT intrusion detection data, followed by the generation of adversarial examples using established perturbation techniques and an assessment of the extent to which these adversarial inputs degraded each model's classification performance. The findings revealed a clear trade-off between the two architectures examined: the feedforward neural network achieved higher baseline classification accuracy under normal conditions, whereas the self-normalising neural network demonstrated comparatively greater resilience when subjected to adversarial perturbation, despite its somewhat lower accuracy in the absence of adversarial interference. This result is significant in that it demonstrates that classification accuracy alone, as reported under standard evaluation conditions, does not fully capture a model's suitability for real-world deployment, where an adversary may actively attempt to circumvent detection. The principal limitation of this study, from the perspective of the present work, is that it does not incorporate any explainability mechanism into either of the examined architectures, nor does it explore whether interpretable, feature-attribution-based explanations might themselves assist an analyst in recognising when a classification has potentially been influenced by adversarial manipulation, a possibility that the LIME, SHAP, and Integrated Gradients modules developed in this project could plausibly support, since an unusual or inconsistent pattern of feature attribution may itself serve as an indirect indicator of adversarial interference. While adversarial robustness is not the primary focus of the present thesis, the findings of Ibitoye et al. are relevant in highlighting that model reliability extends beyond accuracy metrics obtained under ordinary test conditions, a consideration that motivates the discussion of limitations and future scope presented in the concluding chapter of this work.

Considered collectively, the literature examined across Sections 2.2 to 2.6 reveals a consistent and clearly defined set of research gaps. Across the foundational hybrid architectures reviewed in Theme A, the individually refined deep learning approaches reviewed in Theme B, and the classification-focused study reviewed in Theme D, detection performance has been pursued extensively and successfully, with numerous studies reporting high classification accuracy

across a range of botnet-related attack categories. However, this pursuit of accuracy has, with near uniformity, proceeded independently of any corresponding effort toward model transparency, a pattern explicitly confirmed by the survey findings of McDermott et al. and left unaddressed even in studies, such as that of Ali et al., whose architectural design most closely resembles the stacked approach adopted in the present work. Similarly, the dataset and benchmarking contributions reviewed in Theme C, while essential to enabling meaningful comparative evaluation, are themselves methodology-agnostic and offer no inherent solution to the interpretability gap; and the robustness concerns raised by Ibitoye et al. in Theme E demonstrate that accuracy-focused evaluation alone may additionally overlook important questions of model reliability under adversarial conditions. No single study reviewed in this chapter combines architectural sophistication, comprehensive multi-class attack coverage, and a rigorously integrated explainability framework within one unified system. The proposed XAI-Based Stacked Attention Deep Learning Framework is designed specifically to address this convergence of gaps: it adopts the hybrid, stacked architectural philosophy validated by Ali et al. and extends it through the addition of an attention mechanism absent from all reviewed studies; it evaluates performance across the full breadth of attack categories present in UNSW-NB15, addressing the narrower evaluation scope observed in several individually reviewed works; and, most significantly, it integrates LIME, SHAP, and Integrated Gradients directly into the detection pipeline, ensuring that the interpretability limitation identified as a near-universal characteristic of the existing literature is treated not as an afterthought but as a foundational design requirement of the proposed system.

2.7 Comparative Summary of Reviewed Literature

Table 2.1 presents a consolidated comparison of the fourteen studies reviewed in this chapter, summarising the method adopted, the dataset used for evaluation, the key findings reported, and the principal limitation identified in each case.

Table 2.1: Comparative Summary of Reviewed Literature

Ref.	Author(s)	Method	Dataset	Key Findings	Limitation
[1]	Ali et al.	ACLR (ANN + CNN + LSTM + RNN hybrid)	IoT Network Traffic	Achieved over 96% accuracy with strong ROC-AUC through architectural stacking	No explainability mechanism
[21]	Alzahrani & Bamhdi	Hybrid deep learning model	IoT Dataset	Improved accuracy over single-model baselines via combined architecture	High computational cost; no interpretability
[24]	Popoola et al.	Deep feature learning-based IDS	IoT Traffic	Strong generalisation across multiple attack types	No discussion of prediction reasoning
[26]	Popoola et al.	Stacked recurrent neural network	Smart Home IoT Traffic	Layered recurrent stacking improved detection over shallow RNNs	Limited to recurrent processing; no attention or XAI
[3]	Shahhosseini et al.	Deep learning on raw traffic data	Network Traffic	Reduced dependence on manual feature engineering	Limited interpretability of raw-traffic predictions
[4]	Homayoun et al.	BoTShark (deep learning on C&C flow patterns)	Botnet Traffic Flows	Effective detection based on communication behaviour	Generalisability tied to trained botnet families; no XAI
[10]	Liu et al.	Deep learning IDS adapted	IoT Traffic	Demonstrated successful transfer	Narrow attack category

		for IoT		of DL methods to IoT context	coverage; black-box output
[16]	Haq & Khan	DNNBoT (multi-class DNN classifier)	Botnet Traffic (multi-category)	Strong multi-class botnet categorisation performance	No attention mechanism or explainability layer
[23]	Chen et al.	CNN-based convolutional feature extraction	Botnet Traffic	Convolutional features improved detection effectiveness	No recurrent extension or interpretability
[30]	Koroniotis et al.	Bot-IoT dataset development	Bot-IoT	Created realistic, multi-category IoT botnet benchmark dataset	Dataset-only contribution; no detection model proposed
[12]	Sriram et al.	Flow-based deep learning detection	IoT Network Flows	Flow-level features sufficient for accurate detection	Narrow attack category scope; no explainability
[11]	McDermott et al.	Survey of DL in cybersecurity	Multiple (survey)	Confirmed black-box nature is pervasive across DL-based IDS literature	No proposed solution or XAI technique evaluated
[18]	Ahmed et al.	Deep learning classification model	Botnet Traffic	Achieved strong multi-category classification accuracy	No interpretability mechanism
[2]	Ibitoye et al.	FNN and SNN under adversarial evaluation	IoT IDS Dataset	SNN more robust to adversarial perturbation than FNN	No explainability; robustness–accuracy trade-off unresolved

2.8 Summary of Literature Survey

The literature reviewed across this chapter reflects a substantial and technically diverse body of research directed toward the detection of botnet attacks within IoT network environments. Studies examined under Theme A demonstrated that hybrid and ensemble architectures, combining ANN, CNN, LSTM, and RNN components, consistently outperform individual deep learning models, with Ali et al.'s ACLR framework in particular establishing the closest architectural precedent to the model developed in this project. Theme B extended this understanding by showing that individually refined architectures, whether operating on raw traffic, convolutional feature representations, or flow-based inputs, remain capable of strong detection performance even without full hybridisation. Theme C established the empirical foundation upon which such models are evaluated, with Koroniotis et al.'s Bot-IoT dataset and the flow-based methodology examined by Sriram et al. underscoring the importance of realistic, well-structured benchmark data. Theme D confirmed, through both a broad survey and an individually reviewed classification study, that model interpretability remains a largely unaddressed concern across the field, while Theme E introduced the additional consideration of adversarial robustness, demonstrating that accuracy under standard evaluation conditions does not necessarily reflect reliability under adversarial pressure. Taken together, these fourteen studies illustrate a field that has matured considerably in terms of detection accuracy and architectural sophistication, yet remains largely unresolved with respect to transparency, interpretability, and the practical trust required for real-world security deployment.

2.9 Research Gap

Based on the literature reviewed in this chapter, the following specific gaps have been identified:

Existing hybrid deep learning architectures, including the closely related ACLR model, achieve strong classification accuracy but incorporate no attention-based mechanism capable of selectively weighting the most relevant traffic features during classification. Individually refined architectures such as convolutional or recurrent models operating in isolation demonstrate the viability of their respective techniques but do not combine spatial feature extraction, selective attention, and sequential modelling within a single unified pipeline. Benchmark dataset contributions, while essential, are inherently methodology-agnostic and do not, by themselves, resolve the interpretability limitations of the detection models

evaluated against them. Almost universally, across hybrid, individual, and classification-focused studies alike, no mechanism is provided for explaining why a specific traffic sample was classified as malicious or benign, a limitation explicitly confirmed by the survey findings of McDermott et al. and left unaddressed even in the highest-performing architectures reviewed. Finally, considerations of adversarial robustness, where examined at all, are treated entirely separately from questions of interpretability, despite the plausible connection between anomalous feature attribution patterns and adversarially manipulated inputs. No study reviewed in this chapter simultaneously addresses architectural sophistication through attention-enhanced stacking, comprehensive multi-class attack coverage, and rigorous, multi-technique explainability. This convergence of unresolved gaps directly motivates the design of the proposed XAI-Based Stacked Attention Deep Learning Framework, which is developed specifically to combine these three dimensions within a single, cohesive detection system.

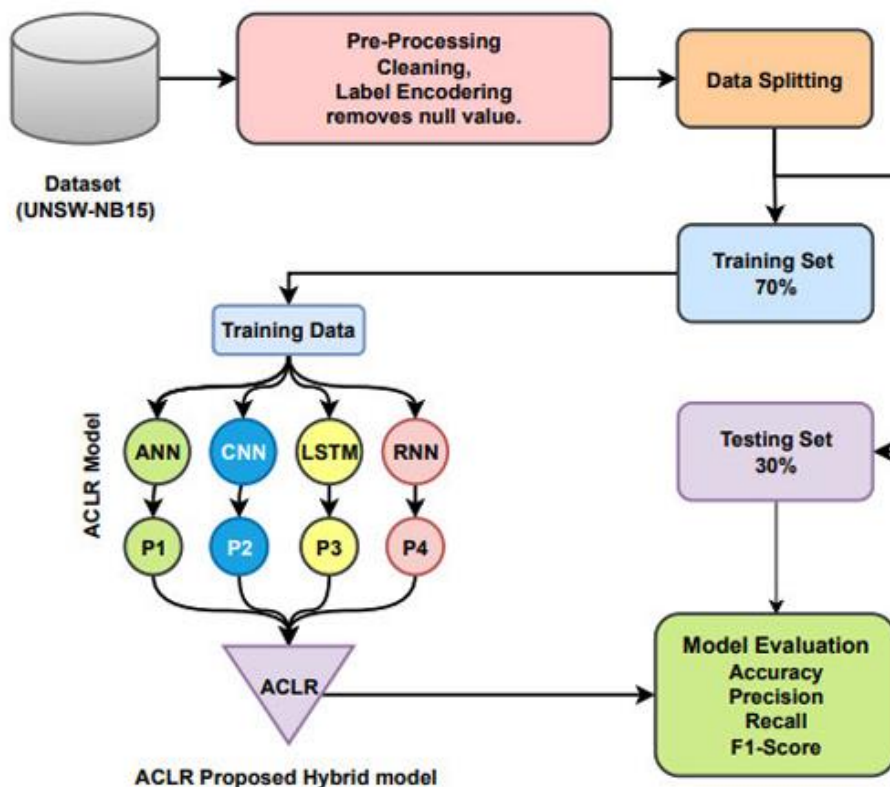


FIGURE 1. Architecture of the proposed approach.

Figure 2.1 Architecture of the proposed approach



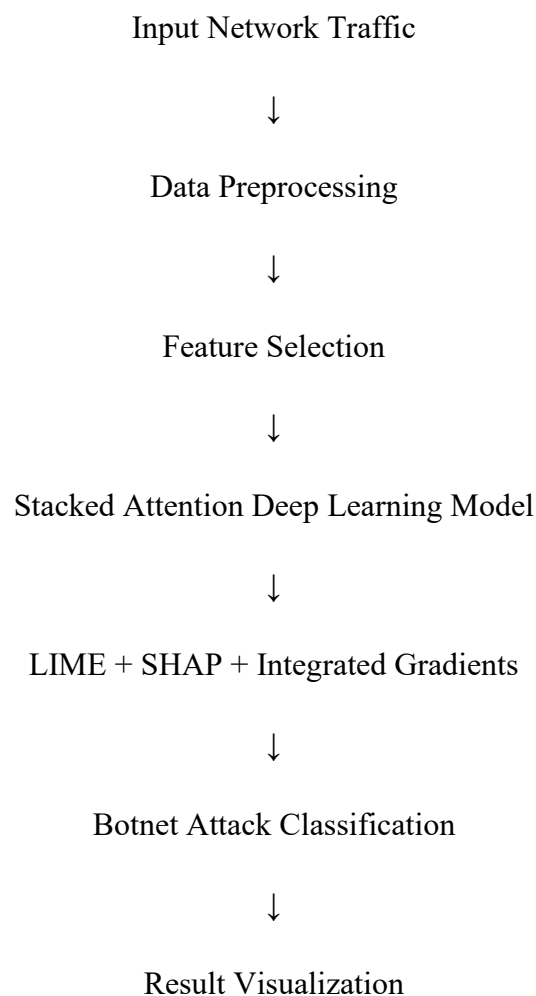
SYSTEM DESIGN

CHAPTER 3

SYSTEM DESIGN

3.1 Importance of Design

System design is the foundation of any software development process. It defines the architecture, components, modules, interfaces, and data flows required to build a robust and scalable system. In the context of IoT security, where devices are heterogeneous and threats evolve rapidly, a strong system design ensures modularity, ease of integration, and future scalability. Designing a hybrid machine learning-based intrusion detection system (IDS) requires careful mapping of data flows, model integration, and user-system interactions to ensure effective detection with minimal resource consumption.



3.2 UML Diagrams

3.2.1 Use Case Diagram

This diagram outlines the key interactions between users (e.g., network administrators) and the system, showing functionalities such as uploading IoT traffic data, training the model, and detecting botnet attacks.

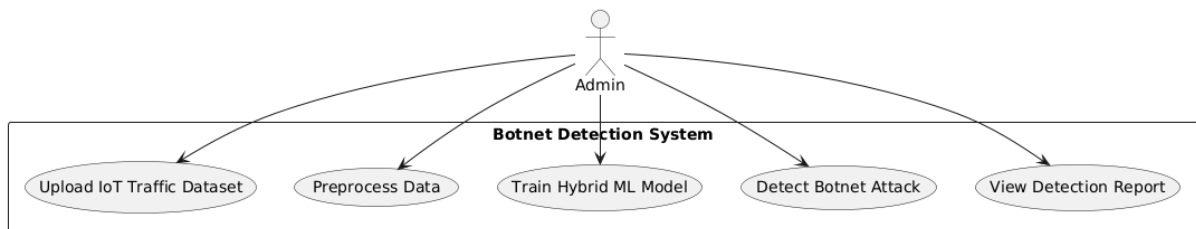


Figure 3.1: Use Case Diagram

3.2.2 Class Diagram

The class diagram represents the structure of the system. It defines main classes like DatasetManager, FeatureExtractor, HybridModel, and DetectionEngine with their attributes and methods.

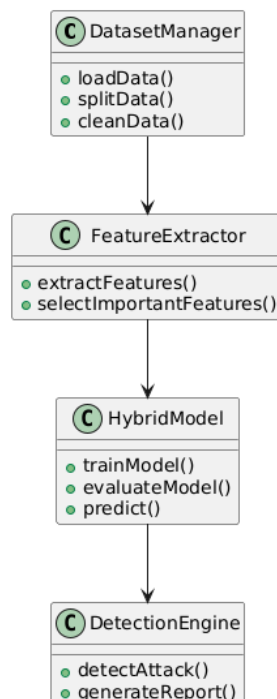


Figure 3.2: Class Diagram

3.2.3 Sequence Diagram

This diagram describes the flow of operations, from dataset upload to detection and reporting.

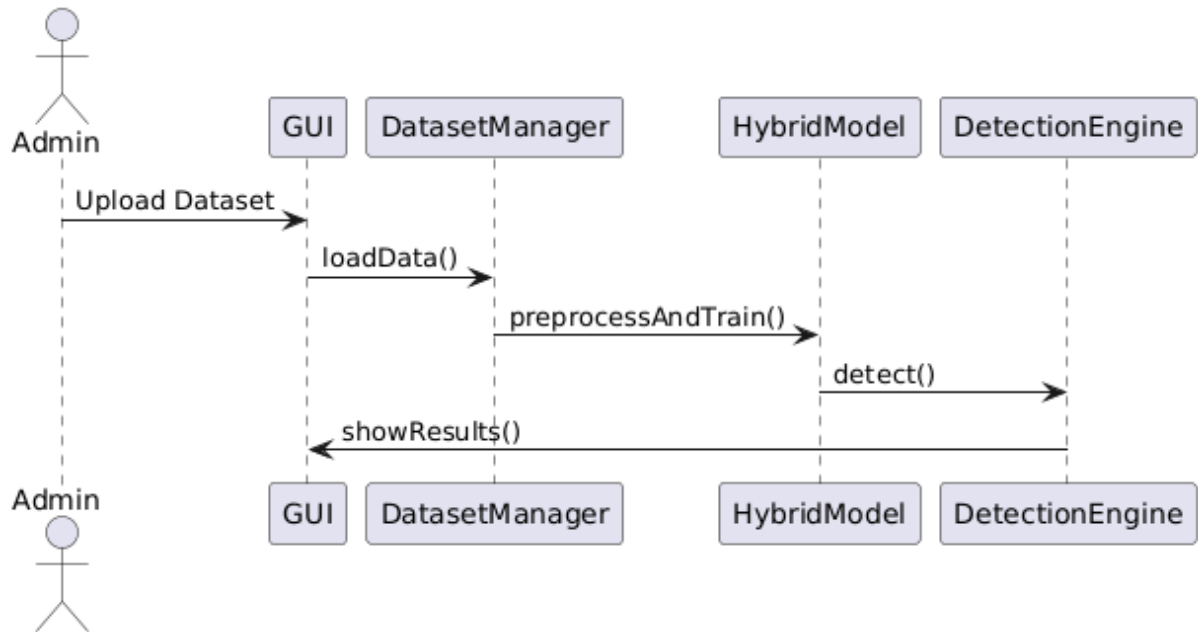


Figure 3.3: Sequence Diagram

3.2.4 Activity Diagram

The activity diagram shows the operational flow of the botnet detection system from start to finish.

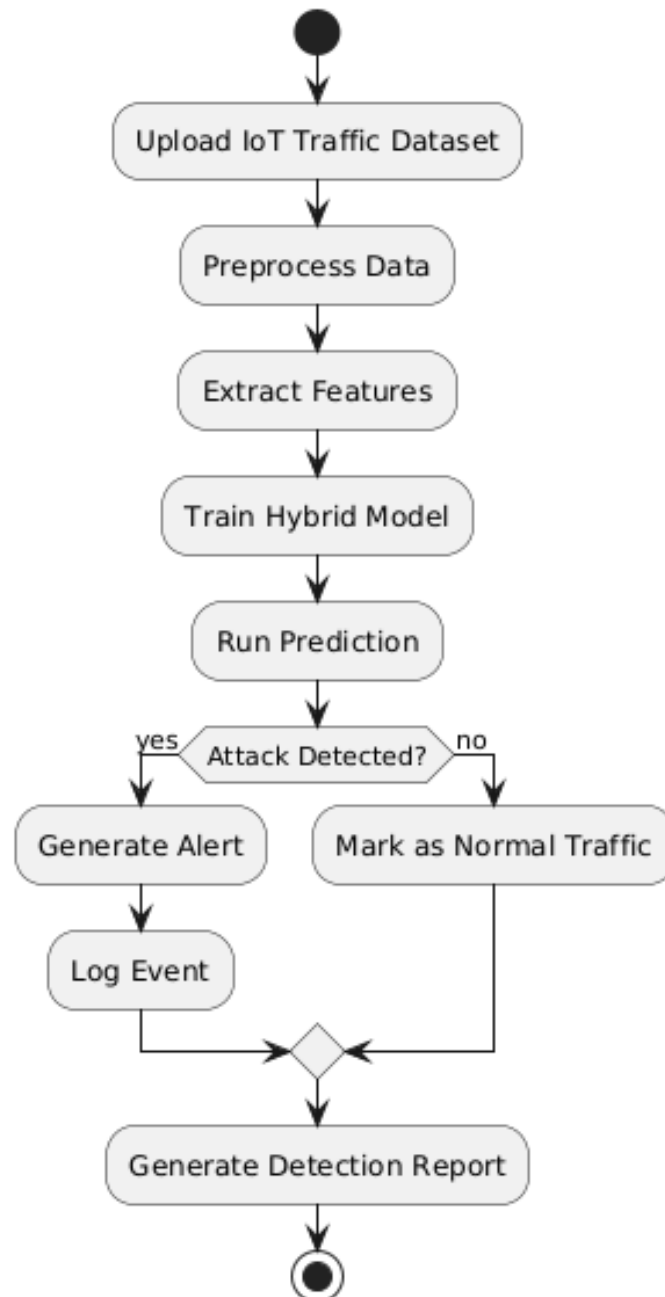


Figure 3.4: Activity Diagram

3.2.5 System Architecture

The proposed system architecture consists of network traffic acquisition, data preprocessing, feature extraction, stacked attention-based deep learning classification, explainable AI analysis, and attack prediction modules. Initially, network traffic data is collected and preprocessed through normalization and feature selection techniques. The processed features are then provided to the Stacked Attention Deep Learning Model, which learns complex traffic patterns and identifies malicious botnet activities. To improve model transparency, LIME, SHAP, and Integrated Gradients are applied to explain prediction decisions. Finally, the system classifies network traffic as normal or malicious and generates the corresponding results for security analysis. The proposed system architecture consists of six major stages: network traffic acquisition, data preprocessing, feature extraction, stacked attention-based deep learning classification, explainability analysis, and attack prediction. Initially, network traffic records are collected and preprocessed using normalization and feature selection techniques. The processed data is then supplied to the Stacked Attention Deep Learning Model, which learns complex traffic behavior and identifies malicious botnet activities. To enhance model transparency, LIME, SHAP, and Integrated Gradients are employed to explain prediction outcomes. Finally, the system classifies traffic as normal or malicious and displays the corresponding results to the user.

System Architecture - Hybrid ML Botnet Detection

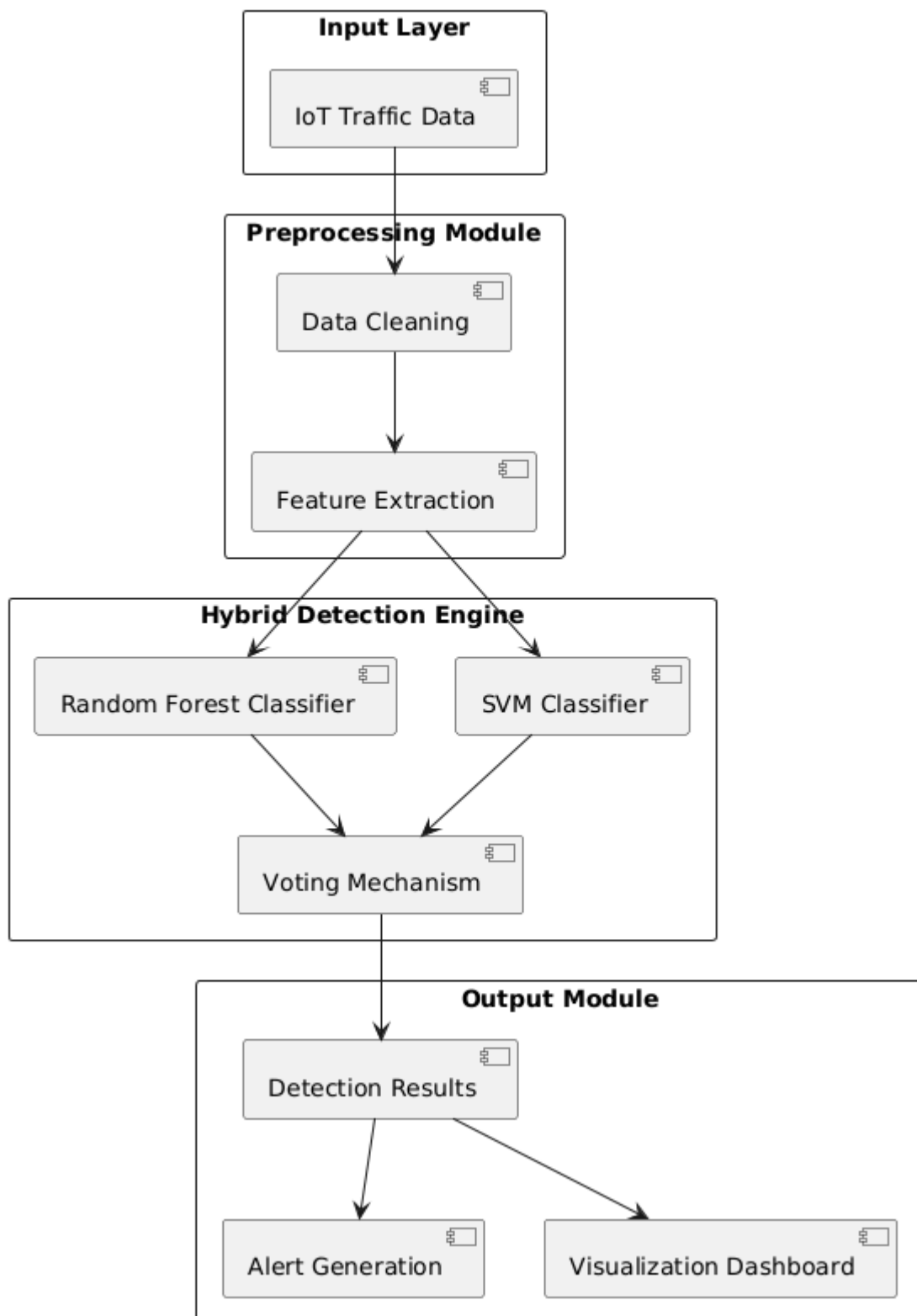


Figure 3.5: System Architecture

3.2.6 Deployment Diagram:

The Deployment Diagram illustrates the physical components of the system, including **servers, IoT devices, and cloud infrastructure**. It ensures proper allocation of resources for processing large-scale network data and executing deep learning models effectively.

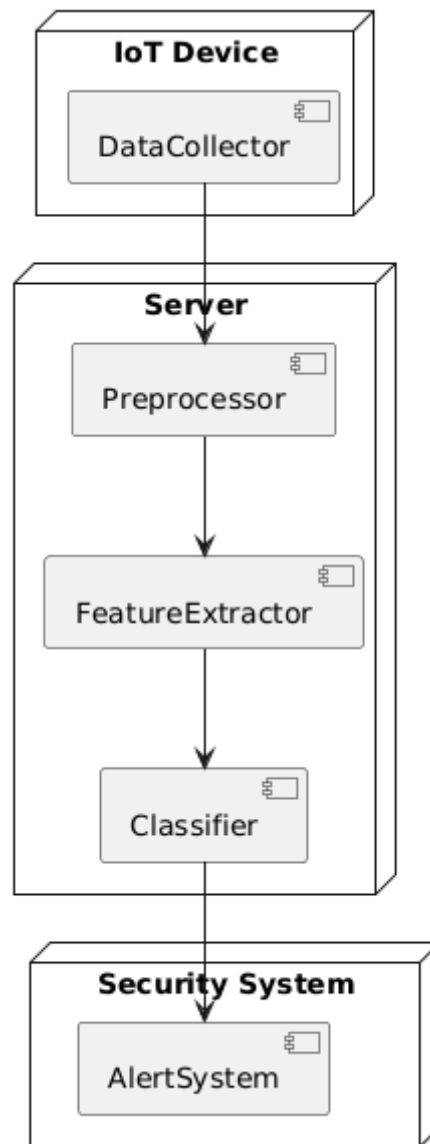


Figure 3.6: Deployment Diagram

3.2.7 Component Diagram the Component Diagram showcases the modular structure of the system, depicting different software components such as **Data Processing Module, Feature Extraction Engine, ML Model Training, and Alert System**. This ensures better system maintainability and scalability.

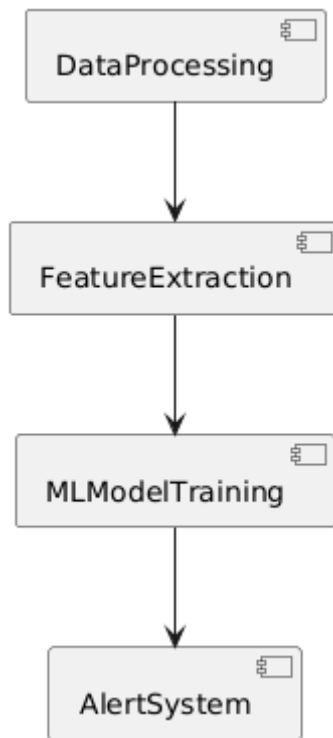


Figure 3.7: Component Diagram

3.3 Functional Requirement

Hardware Requirements

- Processor: Intel Core i5/i7 or AMD Ryzen 5/7
- RAM: 8 GB Minimum (16 GB Recommended)
- Storage: 50 GB Free Disk Space
- GPU: NVIDIA GPU (Optional)

Software Requirements

- Operating System: Windows 10/11
- Programming Language: Python 3.10+
- IDE: Jupyter Notebook / VS Code
- Libraries: TensorFlow, Keras, Scikit-Learn, Pandas, NumPy, Matplotlib, LIME, SHAP

IMPLEMENTATION

CHAPTER 4

IMPLEMENTATION

4.1 Overall Implementation Overview

The implementation phase of this project translates the conceptual design discussed in Chapter 3 into a functioning software system capable of detecting botnet attacks in IoT network traffic while offering interpretable explanations for its predictions. This phase encompasses the complete pipeline of activities beginning with the acquisition of network traffic data and concluding with the generation of human-understandable prediction explanations through Explainable Artificial Intelligence techniques. The implementation was carried out using the Python programming language, supported by widely adopted deep learning and data science libraries, ensuring compatibility, reproducibility, and ease of experimentation.

The overall workflow adopted for this project follows a systematic sequence of stages. It begins with the acquisition of the benchmark network intrusion dataset, followed by a thorough preparation and cleaning process to ensure that the data supplied to the learning algorithms is consistent and free of anomalies that could adversely affect model performance. Once the dataset has been prepared, it undergoes a series of preprocessing operations, including categorical label encoding and feature scaling, which transform the raw network traffic attributes into a numerical representation suitable for deep learning models. The prepared data is subsequently partitioned into training and testing subsets, enabling the models to learn underlying traffic patterns while reserving a portion of the data for unbiased performance evaluation.

Following data preparation, the project proceeds to the construction and training of multiple deep learning architectures, namely Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), and Long Short-Term Memory (LSTM) networks, which are subsequently combined into a stacked ensemble. This stacked architecture is further enhanced through the incorporation of an attention mechanism, allowing the model to dynamically emphasise the most relevant traffic features during classification. Once trained, the model is used to classify unseen network traffic records as either normal or belonging to a specific botnet attack category. To address the inherent lack of transparency in deep learning-based decision-making, the trained model's predictions are analysed using three complementary explainability techniques, namely LIME, SHAP, and Integrated Gradients, each of which highlights the contribution of individual traffic features toward a given prediction. Finally, the trained model

and its associated preprocessing components are integrated into a web-based application that allows authorised users to upload traffic data, obtain classification results, and interact with the system through a structured interface.

The subsequent sections of this chapter describe each of these implementation stages in detail. Section 4.2 discusses the dataset employed in this study, its characteristics, and the process of loading and organising it for experimentation. Section 4.3 elaborates on the preprocessing operations applied to the raw dataset prior to model training. Later sections of this chapter will address the architecture and training of the individual and stacked deep learning models, the implementation of the explainability modules, and the development of the web-based prediction interface.

4.2 Dataset Loading and Preparation

The proposed botnet attack detection framework is trained and evaluated using the UNSW-NB15 dataset, a widely recognised benchmark dataset developed for network intrusion detection research. This dataset was selected because it captures a realistic blend of normal network behaviour alongside a diverse range of contemporary attack types, making it well suited for evaluating a detection system intended for modern IoT-adjacent network environments. Unlike older intrusion detection datasets that primarily represent a narrow set of legacy attack patterns, UNSW-NB15 reflects a broader and more current threat landscape, comprising categories such as Denial of Service, Exploits, Fuzzers, Reconnaissance, Backdoors, Shellcode, Worms, Generic, and Analysis attacks, in addition to normal traffic records. Each record in the dataset is described using a comprehensive set of network flow-based features, including attributes derived from packet timing, protocol behaviour, connection state, and traffic volume, which collectively provide a rich feature space for a deep learning model to learn discriminative patterns between benign and malicious activity.

The dataset was loaded into the working environment using standard data-handling utilities, which allow the network traffic records to be represented in a structured tabular format suitable for subsequent analysis. Upon loading, an initial inspection of the dataset was performed to understand its structure, the distribution of records across the various traffic categories, and the nature of the available features. This inspection also included a visual analysis of the class distribution, which revealed that certain attack categories are represented by considerably fewer records than others, a characteristic that is typical of real-world network traffic and one that has implications for model training, as discussed in later sections of this chapter.

Feature selection for this project was guided by the structure of the dataset itself. Rather than manually curating a subset of features, the framework retains the complete set of flow-based traffic attributes provided by the dataset, excluding only the columns that serve as classification targets. This approach allows the deep learning models to automatically learn which combinations of features are most relevant to distinguishing between normal and malicious traffic, a task for which neural architectures are particularly well suited given their capacity for automatic feature learning. The target label used for classification purposes is the attack category attribute, which identifies the specific type of traffic represented by each record, including the normal class. Handling of this label required converting the descriptive, text-based category names into a numerical representation, since deep learning models operate exclusively on numerical inputs. This transformation preserves the distinct identity of each attack category while making the label compatible with the categorical classification objective used during model training.

Once the target labels had been identified and separated from the descriptive feature set, the dataset was organised into two principal components: a feature matrix containing the numerical network traffic attributes, and a corresponding label vector representing the traffic category associated with each record. This clear separation between features and labels is essential for supervised learning, as it enables the model to learn a mapping between input traffic characteristics and their correct classification without inadvertently using the target information during the learning process. The organised dataset thus formed the foundation upon which all subsequent preprocessing and model training activities were carried out.

4.3 Data Preprocessing

Data preprocessing constitutes a critical stage in the implementation of any deep learning-based system, as the quality and consistency of the input data directly influence the ability of the model to learn meaningful patterns. In the context of this project, several preprocessing operations were applied to the UNSW-NB15 dataset prior to model training to ensure that the resulting data was clean, numerically consistent, and appropriately structured for the deep learning architectures employed.

The first preprocessing step involved data cleaning, during which the dataset was examined for missing or incomplete values. Network traffic datasets of this nature often contain gaps arising from incomplete packet captures or fields that are not applicable to certain protocol types. Any missing values identified within the dataset were addressed by substituting them with a neutral

placeholder value, thereby preventing incomplete records from disrupting the training process while preserving the overall structure and size of the dataset. This step ensures that every record presented to the model contains a complete and well-formed set of feature values.

42 Following data cleaning, label encoding was applied to convert the categorical attack classification labels into a numerical format. Since the dataset originally represents traffic categories using descriptive textual labels such as Normal, Backdoor, or Exploits, these labels were systematically mapped to unique integer identifiers. This encoding preserves the distinctiveness of each category while enabling the classification models, which require numerical target values, to be trained using standard categorical loss functions. The encoded labels were subsequently transformed into a one-hot categorical representation, allowing the models to produce a probability distribution across all possible attack categories during prediction, rather than a single scalar output.

63 Feature scaling was another essential preprocessing operation applied to the dataset. The raw traffic features in UNSW-NB15 vary considerably in magnitude and range, as attributes such as byte counts and packet rates can differ by several orders of magnitude from attributes representing durations or ratios. Without appropriate scaling, features with larger numerical ranges could disproportionately influence the learning process, causing the model to place undue emphasis on attributes with larger raw values rather than those that are genuinely more informative. To address this, a standardisation procedure was applied to the feature set, transforming each attribute to have a comparable scale and distribution. This step is particularly important for the deep learning architectures used in this project, as neural networks are known to converge more efficiently and produce more stable results when trained on normalised input data. Additionally, since certain model architectures within the proposed framework, such as the convolutional and attention-based components, expect input data in a specific multidimensional format, the scaled feature vectors were reshaped accordingly to align with the structural requirements of each architecture.

22 Once cleaning, encoding, and scaling had been completed, the prepared dataset was partitioned into training and testing subsets. This partitioning was performed such that the majority of records were allocated for training the models, while a smaller, held-out portion was reserved exclusively for evaluating their performance on previously unseen data. This division is fundamental to obtaining a realistic assessment of how the proposed framework would perform when applied to new network traffic in a practical deployment scenario, as it prevents the evaluation process from being influenced by data the model has already encountered during training.

Collectively, these preprocessing operations play an indispensable role in the overall implementation of the proposed framework. Deep learning models are highly sensitive to the quality, scale, and structure of their input data, and inadequate preprocessing can lead to slow convergence, unstable training, or degraded classification performance regardless of the sophistication of the underlying architecture. By systematically cleaning, encoding, scaling, and partitioning the UNSW-NB15 dataset, this implementation ensures that the subsequent model training stages, discussed in the following sections of this chapter, are built upon a reliable and well-structured data foundation.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
dur	spkts	dpkts	sbytes	dbytes	rate	sttl	dttl	sload	dload	sloss	dloss	sinpkt	dinpkt	sjit	djitt	swin	stcpb	dtcpb	dv
1.129944	10	8	534	354	15.04499	254	252	3405.479	2194.799	2	1	122.617	150.6929	6633.796	259.9588	255	1.37E+09	1.19E+09	
1.964566	10	8	2516	354	8.653311	254	252	9223.411	1262.365	2	1	218.1742	270.6407	13125.1	469.482	255	1.59E+09	2.48E+09	
2.349783	12	10	888	1370	8.936996	62	252	2771.32	4197.834	2	2	213.6166	241.403	12385.52	352.7233	255	1.79E+09	2.63E+09	
7.00E-06	2	0	104	0	142857.1	254	0	59428572	0	0	0	0.007	0	0	0	0	0	0	0
0.005896	22	24	2054	2478	7632.293	31	29	2660787	3222524	6	6	0.264238	0.227435	16.08844	13.6787	255	1.18E+09	3.33E+09	
0.034952	44	46	2750	27602	2546.349	31	29	615243.8	6180362	7	16	0.804651	0.7642	0	56.55126	255	1.52E+09	1.53E+09	
0.007721	30	26	3624	5578	7123.43	31	29	3630618	5557830	7	7	0.266241	0.27776	14.86209	16.73847	255	2.25E+09	98510618	
0.03338	122	126	7814	15690	7399.64	31	29	1857400	3730617	30	32	0.273025	0.262504	17.52596	15.03498	255	1.4E+09	1.4E+09	
0.001099	2	2	146	178	2729.754	31	29	531392.2	647861.7	0	0	0.01	0.009	0	0	0	0	0	0
0.004408	4	4	568	304	1588.022	31	29	773139.8	413793.1	0	0	1.202667	0.980333	1.69093	1.377917	0	0	0	0
7.200272	232	438	13350	548216	92.91316	31	29	14769.44	607715.9	21	197	31.16853	16.47538	12609.72	9178.188	255	6.22E+08	6.8E+08	
0.001114	2	2	146	178	2692.998	31	29	524237	639138.3	0	0	0.011	0.008	0	0	0	0	0	0
0.007465	14	6	8928	320	2545.211	31	29	8885198	286135.3	4	1	0.574231	1.3678	30.1266	2.566771	255	1.58E+08	1.69E+08	
0.021426	52	40	37274	3276	4247.176	31	29	13649958	1192943	18	8	0.413255	0.53518	22.82292	1.160264	255	8.61E+08	3.01E+09	
0.004966	18	18	4488	2456	7047.926	31	29	6828836	3737414	7	7	0.273118	0.260824	13.49666	0.348445	255	3.79E+09	3.79E+09	
1.101911	122	126	7820	15676	224.156	31	29	56309.45	112909.3	30	32	9.103628	8.810832	568.4524	561.6889	255	3.1E+09	3.12E+09	
1.00E-05	2	0	264	0	100000	60	0	1.06E+08	0	0	0	0.01	0	0	0	0	0	0	0
0.001094	2	2	146	178	2742.23	31	29	533820.9	650822.7	0	0	0.011	0.008	0	0	0	0	0	0
1.004652	12	18	1580	10168	28.86572	31	29	11538.32	76476.23	3	5	91.29782	59.06312	9528.554	7674.383	255	1E+09	1E+09	
0.030644	52	42	37264	3380	3034.852	31	29	9541313	861506.3	18	8	0.594078	0.734463	34.2601	1.552289	255	4.58E+08	4.59E+08	

Figure 4.1: UNSW-NB15 dataset

4.4 ANN Implementation

The first deep learning model incorporated into the proposed framework is a fully connected Artificial Neural Network (ANN), implemented to establish a baseline classification benchmark against which the more advanced architectures discussed later in this chapter can be compared. The ANN was selected as an initial model because it represents the most fundamental form of a deep learning classifier, capable of learning non-linear relationships between the input traffic features and the corresponding attack categories without relying on any assumption regarding spatial or sequential structure within the data.

The architecture of the ANN model was designed as a sequential stack of fully connected layers. The input layer accepts the complete set of preprocessed network traffic features, with the number of input neurons corresponding directly to the number of attributes present in the

dataset. This is followed by a hidden layer responsible for learning intermediate, higher-order representations of the traffic features, followed by a second, comparatively smaller hidden layer employing a rectified linear unit activation function. The use of a rectified linear activation in the hidden layers allows the network to model non-linear decision boundaries while mitigating issues such as vanishing gradients that can arise with alternative activation functions. The final layer of the network is a dense output layer whose number of neurons corresponds to the total number of traffic categories present in the dataset, including the normal class. This output layer employs a softmax activation function, which converts the raw output values into a probability distribution across all possible attack categories, allowing the model to indicate not only its predicted class but also its relative confidence across the remaining classes.

The ANN model was compiled using the Adam optimisation algorithm, which was selected due to its adaptive learning rate properties and its widespread effectiveness in training deep learning models efficiently without requiring extensive manual tuning of the learning rate. The categorical cross-entropy loss function was employed as the training objective, as it is well suited to multi-class classification problems in which the target labels are represented in a one-hot encoded format. During training, the model was presented with the training subset of the dataset over multiple epochs, with the testing subset used concurrently for validation to monitor the model's generalisation performance throughout the training process. A model checkpointing mechanism was employed to automatically preserve the best-performing set of weights observed during training, ensuring that the final evaluated model corresponds to the point at which validation performance was optimal rather than to the state of the network at the final training epoch. Once training was complete, the ANN was used to classify the reserved test records, and its predictions were evaluated using the standard classification metrics adopted throughout this study. The ANN implementation thus serves as an important reference point, allowing the subsequent, more sophisticated architectures to be assessed in terms of the performance improvement they offer over a conventional fully connected baseline.

4.5 CNN Implementation

Following the implementation of the ANN baseline, a Convolutional Neural Network (CNN) was developed to evaluate whether convolutional feature extraction could improve the model's ability to identify discriminative patterns within the network traffic features. Although CNNs are conventionally associated with image data, they have been increasingly applied to tabular

and flow-based network traffic datasets by treating the feature vector of each record as a structured input that convolutional filters can traverse. In this implementation, each preprocessed traffic record was reshaped from its original one-dimensional feature vector into a multidimensional format compatible with convolutional layers, effectively presenting the feature set to the network as a narrow, structured grid rather than a flat list of values.

The CNN architecture begins with a convolutional layer employing a bank of filters with a compact receptive field, which allows the network to learn localised, feature-wise transformations across the input attributes. This is followed by a pooling layer, which reduces the dimensionality of the learned feature maps while retaining the most salient information extracted by the preceding convolutional operation. A second convolutional layer, using a smaller number of filters, is then applied to further refine the extracted representations, followed by an additional pooling operation. The resulting feature maps are subsequently flattened into a single vector, converting the spatially structured output of the convolutional layers back into a form suitable for classification. This flattened representation is passed to a dense output layer employing a softmax activation function, consistent with the classification approach used in the ANN model, enabling the CNN to produce a probability distribution across all traffic categories.

The rationale for incorporating a CNN into the proposed framework lies in its capacity for automatic hierarchical feature extraction. Rather than relying solely on the raw traffic attributes as presented, the convolutional layers enable the model to learn transformed feature representations that may better capture subtle relationships between attributes associated with malicious behaviour, such as combinations of packet timing and connection statistics. As with the ANN, the CNN was compiled using the Adam optimiser and the categorical cross-entropy loss function, and was trained over multiple epochs using the same batch size and validation strategy, with checkpointing employed to retain the best-performing model weights. The consistent training configuration across models ensures that performance differences observed between architectures can be attributed to differences in their structural design rather than to variations in the training procedure.

4.6 LSTM Implementation

The third individual architecture implemented in this project is a Long Short-Term Memory (LSTM) network, a specialised form of recurrent neural network capable of learning dependencies across sequential input data. Although individual network traffic records in the

UNSW-NB15 dataset are not inherently sequential in the manner of time-series data, the feature vector associated with each record can nonetheless be interpreted as an ordered sequence of attribute values, allowing an LSTM to model potential inter-feature dependencies that a purely feed-forward architecture such as the ANN may not capture. To accommodate this interpretation, the preprocessed feature vectors were reshaped into a three-dimensional format suitable for sequential processing prior to being supplied to the LSTM layer.

The LSTM architecture implemented in this project consists of a recurrent LSTM layer responsible for learning temporal or sequential dependencies within the reshaped feature representation. Following the recurrent layer, a dropout layer was introduced with a comparatively high dropout rate, serving as a regularisation mechanism to reduce the risk of overfitting by randomly deactivating a proportion of neurons during each training iteration. This is particularly important for recurrent architectures, which are generally more susceptible to overfitting on limited or imbalanced training data than simpler feed-forward networks. Following the dropout layer, a fully connected dense layer employing a rectified linear activation function was included to further refine the representation learned by the LSTM before classification. The network concludes with a dense output layer employing a softmax activation function, consistent with the output configuration adopted for the ANN and CNN models, thereby ensuring uniformity in how predictions are interpreted across all three baseline architectures.

The LSTM model was compiled using the categorical cross-entropy loss function in conjunction with the Adam optimiser, and was trained using the same batch size, number of epochs, and validation configuration applied to the preceding models, with the best-performing weights preserved through checkpointing. The inclusion of the LSTM within the proposed framework allows the study to examine whether sequential modelling of traffic attributes offers any advantage over purely spatial or fully connected approaches. Collectively, the ANN, CNN, and LSTM models described in this and the preceding sections form the individual building blocks upon which the proposed stacked and attention-enhanced architecture, discussed in the following section of this chapter, is constructed.

4.7 Stacked Attention Model Implementation

Building upon the individual ANN, CNN, and LSTM architectures discussed in the preceding sections, the core contribution of this project lies in the design and implementation of a stacked attention-based deep learning model that integrates convolutional feature extraction, an

attention mechanism, and recurrent sequence modelling within a single unified architecture. This model was developed with the objective of combining the complementary strengths of its constituent components while addressing the limitations observed when each architecture is employed in isolation.

52 The overall architecture of the proposed model begins with the same reshaped, standardised feature representation used for the CNN implementation, allowing convolutional operations to be applied to the preprocessed traffic attributes. A convolutional layer, followed by a pooling operation, is employed at the initial stage of the network to extract localised patterns from the input feature set, in a manner consistent with the CNN model described in Section 4.5. A second convolutional and pooling stage further refines these extracted representations, condensing the traffic attributes into a compact set of learned features. This convolutional front-end serves as the feature extraction component of the stacked model, responsible for transforming the raw, standardised traffic attributes into a higher-level representation that emphasises locally significant patterns within the data, before this representation is passed onward for sequential and attention-based processing.

Following the convolutional feature extraction stage, the flattened output is expanded into a repeated sequence structure, effectively transforming the single feature vector produced by the convolutional layers into a short sequence suitable for processing by attention and recurrent layers. This repeated sequence is passed through a dedicated attention layer, which forms the central novelty of the proposed architecture. Rather than treating every extracted feature with equal importance, the attention mechanism learns a set of weights that indicate the relative significance of different positions within the sequence representation, allowing the network to dynamically emphasise the traffic characteristics that are most influential in distinguishing between normal and malicious behaviour, while suppressing the influence of less relevant information. This selective weighting is intended to mirror, to a certain extent, the reasoning process of a human security analyst, who instinctively focuses attention on the most suspicious traffic indicators rather than considering all available attributes with equal weight.

50 The output of the attention layer is subsequently passed to a Long Short-Term Memory layer, integrating the sequence-aware, attention-weighted representation with the recurrent modelling capability discussed in Section 4.6. This LSTM stage allows the network to capture any residual dependencies across the attention-weighted feature sequence, further refining the learned representation before classification. The output of the LSTM layer is passed through a fully connected dense layer, which consolidates the learned representation into a compact intermediate form, before a final dense output layer, employing a softmax activation function,

produces the **classification probabilities** across all traffic categories. This final layer maintains consistency with the output configuration adopted throughout the individual baseline models, ensuring that the classification results produced by the stacked attention model remain directly comparable to those obtained from the ANN, CNN, and LSTM architectures. **The model was compiled using the Adam optimiser and categorical cross-entropy loss function, and trained** under the same batch size, epoch count, and validation methodology applied to the earlier models, with checkpointing used to retain the best-performing configuration of weights.

The decision to adopt this particular architecture was motivated by the recognition that botnet attack traffic often manifests through subtle, feature-specific irregularities that are not uniformly distributed across all available attributes. A conventional CNN or LSTM applied independently is capable of extracting either spatial or sequential patterns, but neither architecture inherently distinguishes which of the extracted patterns are most relevant to the classification decision. By introducing an attention mechanism between the convolutional feature extraction stage and the recurrent modelling stage, the proposed architecture is able to combine the pattern-extraction strengths of convolutional layers with a mechanism for selectively prioritising the most informative patterns, before passing this refined representation through a recurrent layer for final consolidation. This design choice reflects a deliberate attempt to move beyond simple architectural stacking toward a model that can, at least partially, indicate which aspects of the traffic representation it considers most important, laying the foundation for the explainability techniques discussed in the following section.

In comparison to the individual ANN, CNN, and LSTM models, the stacked attention architecture offers several conceptual advantages. Where the ANN operates directly on raw traffic attributes without any dedicated feature extraction stage, the proposed model benefits from convolutional pre-processing of the feature space. Where the standalone CNN treats all extracted features with uniform importance during classification, the proposed model introduces an explicit weighting mechanism that allows certain features to exert greater influence on the final prediction. Where the standalone LSTM processes the raw feature sequence without any prior spatial refinement, the proposed model supplies the recurrent layer with a convolutionally extracted and attention-weighted representation, potentially easing the burden placed on the recurrent component. By combining these mechanisms within a single pipeline, the stacked attention model is designed to achieve more effective traffic classification than any of its constituent architectures operating independently, while simultaneously providing a structural basis for interpreting which traffic characteristics most strongly influence its predictions.

4.8 Explainable AI (XAI) Implementation

While the stacked attention model described in Section 4.7 is designed to achieve strong classification performance, its internal decision-making process, like that of most deep learning architectures, remains difficult for a human analyst to interpret directly from its learned weights alone. This lack of transparency presents a significant challenge in cybersecurity applications, where analysts must be able to justify and validate automated decisions, particularly when those decisions influence critical actions such as isolating a device or blocking network traffic. To address this challenge, three complementary Explainable Artificial Intelligence techniques, namely LIME, SHAP, and Integrated Gradients, were incorporated into the implementation to provide human-interpretable explanations of the model's predictions.

The implementation of LIME within this project involves generating a local, interpretable approximation of the stacked attention model's behaviour around a specific traffic record selected for explanation. Because LIME operates on tabular data by perturbing the feature values of the selected instance and observing the corresponding changes in the model's predicted probabilities, a dedicated prediction wrapper function was implemented to bridge the format expected by the LIME library and the multidimensional input structure required by the stacked attention model. This wrapper transforms the perturbed tabular samples generated by LIME into the appropriately scaled and reshaped format before passing them to the trained model, and returns the resulting class probabilities back to the LIME explainer. Using these perturbed samples and their associated predictions, LIME fits a simple, locally weighted linear model that approximates the behaviour of the complex stacked attention model in the immediate vicinity of the selected traffic record, thereby identifying which individual features most strongly influenced that specific prediction.

The SHAP implementation follows a conceptually related but methodologically distinct approach, grounded in cooperative game theory. A kernel-based SHAP explainer was employed, using a small representative sample of training records as a background reference distribution against which the contribution of each feature is measured. For a given set of test records, SHAP systematically evaluates how the model's prediction changes as different combinations of features are included or excluded relative to this background distribution, attributing to each feature a Shapley value that reflects its average marginal contribution to the prediction across all considered feature combinations. As with LIME, a dedicated prediction function was implemented to ensure that the tabular samples generated internally by the SHAP explainer are correctly scaled and reshaped before being submitted to the stacked attention

model. The resulting SHAP values are visualised using summary plots that aggregate feature contributions across multiple records, offering both a global perspective on which features are generally most influential and a local perspective on individual predictions.

The Integrated Gradients technique was implemented to provide a complementary, gradient-based form of explanation specifically suited to differentiable models such as the stacked attention network. This method computes feature attributions by comparing a selected test instance against a defined baseline input, typically represented by a zero-valued reference, and accumulating the gradients of the model's output with respect to each input feature along a series of interpolated steps between the baseline and the actual instance. The resulting attribution values indicate the extent to which each feature contributed positively or negatively to the model's prediction for that specific instance, with the direction and magnitude of each value offering direct insight into the model's reasoning at the level of individual traffic attributes.

Collectively, these three explainability techniques were integrated into the implementation as a post-hoc analysis layer that operates alongside, rather than within, the trained stacked attention model, allowing the classification and explanation processes to remain conceptually and computationally separate. This separation ensures that the explainability modules can be applied to any given prediction without altering the underlying trained model. The inclusion of explainability is of particular importance in the context of botnet attack detection, where security analysts require not only an indication that traffic has been classified as malicious, but also a clear understanding of which specific traffic characteristics led to that classification, in order to validate the decision, prioritise response actions, and identify potential false alarms. By offering three distinct yet complementary perspectives, local surrogate modelling through LIME, game-theoretic feature attribution through SHAP, and gradient-based attribution through Integrated Gradients, the proposed framework enables analysts to cross-verify the reasoning behind a given prediction from multiple angles, thereby substantially improving the transparency, trustworthiness, and practical usability of the deep learning-based detection system within real-world cybersecurity operations.

4.9 Flask Web Application Implementation

To make the trained stacked attention model accessible to end users in a practical and usable form, the proposed framework was integrated into a web-based application developed using the Flask framework, a lightweight and widely adopted Python web development library. The

web application serves as the user-facing layer of the overall system, allowing authorised individuals to interact with the underlying detection model without requiring any direct interaction with the training pipeline, the dataset, or the model files themselves.

7 The architecture of the web application follows a conventional client-server design, in which the Flask backend hosts a set of route handlers responsible for managing user requests, while rendered HTML templates present the corresponding interface to the user's browser. Each functional aspect of the system, including user registration, authentication, dataset upload, prediction, and result display, is implemented as a distinct route within the application, allowing the system to maintain a clear separation of concerns between its various responsibilities. This modular routing structure also facilitates future extension of the application, as additional functionality can be incorporated as new routes without disrupting the existing components.

4 User authentication within the application is implemented through a login mechanism that verifies user-supplied credentials against records stored in a backend database prior to granting access to the detection functionality. A complementary registration module allows new users to create an account by supplying identifying details such as a username, password, and contact information, which are subsequently stored for future authentication. Access to the core prediction functionality of the system is restricted to users who have successfully completed this authentication process, ensuring that the detection capability is not exposed to unauthorised parties and that a defined access-control boundary exists between the public-facing entry point of the application and its analytical functionality.

Once authenticated, users are able to interact with the dataset upload module, which allows a network traffic file to be submitted to the system for analysis. Upon receiving the uploaded data, the application performs the same preprocessing operations applied during model development, namely handling of missing values and feature scaling, ensuring that the uploaded records are transformed into a format consistent with the data used to train the stacked attention model. This consistency between training-time and inference-time preprocessing is essential, as any discrepancy between the two would compromise the validity of the resulting predictions.

The prediction module represents the analytical core of the web application. Upon receiving a preprocessed traffic sample, the trained stacked attention model, reconstructed and loaded with its previously saved weights, is used to generate a classification for each submitted record, identifying the traffic as either normal or as belonging to a specific botnet attack category. The result visualisation component then presents these classifications to the user in a structured,

readable format, distinguishing between normal and malicious records through clear visual indicators, allowing a user to quickly identify which submitted traffic samples require further attention or investigation.

Database connectivity within the application is managed through a Python-based database connector, which facilitates communication between the Flask backend and a relational database used to persist user account information. This database stores the registration details of application users and is queried during the authentication process to validate login credentials. The use of a relational database for this purpose ensures that user information is retained across sessions and that the authentication mechanism operates independently of the model training and prediction components of the system.

With regard to security considerations, the web application incorporates a session-based access-control mechanism that restricts the prediction and result-viewing functionality to users who have successfully authenticated through the login module, thereby preventing unrestricted public access to the detection system. As with any prototype system developed primarily to demonstrate the feasibility of integrating a deep learning-based detection model within an interactive application, the current implementation represents a functional baseline rather than a production-hardened deployment. Strengthening credential storage through password hashing, adopting parameterised database queries, and enforcing stricter input validation on uploaded files are recognised as important directions for future refinement of the application, and are discussed further as part of the future scope of this project.

4.10 Prediction Workflow

The complete prediction workflow of the proposed system begins at the point at which an authenticated user submits a network traffic file through the web application interface. Upon submission, the uploaded data is first passed through the same preprocessing sequence described in Section 4.3, ensuring that missing values are handled appropriately and that the numerical traffic attributes are scaled using the same transformation parameters established during the original model training process. This step guarantees that the characteristics of the incoming data are consistent with those of the data on which the stacked attention model was originally trained, a requirement that is essential for the model to produce reliable predictions. Following preprocessing, the transformed traffic records are reshaped into the multidimensional structure required by the stacked attention model's convolutional and recurrent components, mirroring the reshaping operations applied during model development.

The reshaped data is then supplied to the trained stacked attention model, which processes each record through its convolutional feature extraction layers, its attention mechanism, and its recurrent LSTM component, ultimately producing a probability distribution across all possible traffic categories for each submitted record. The category associated with the highest predicted probability is selected as the final classification outcome for that record, indicating whether the traffic is considered normal or representative of a specific botnet attack type.

Once the classification results have been generated, they are presented to the user through the result visualisation component of the web application, allowing the outcome of the analysis to be reviewed directly within the browser interface. In parallel with this classification workflow, selected predictions may be further examined through the explainability modules described in Section 4.8. When explanation is required for a given prediction, the corresponding traffic record is supplied to the LIME, SHAP, or Integrated Gradients modules, each of which analyses the model's behaviour with respect to that specific instance and produces a corresponding explanation output, indicating the traffic features that most strongly influenced the classification decision. This explanatory output is generated independently of the primary classification workflow, allowing the system to provide both an immediate detection result and a deeper, more transparent justification for that result when such insight is required by the user. Together, these stages form a continuous pipeline extending from raw data submission through preprocessing, classification, and, where applicable, explanation, culminating in a complete and interpretable detection outcome.

4.11 Module Interaction

The overall effectiveness of the proposed framework depends not only on the individual components described throughout this chapter, but also on the manner in which these components interact to form a cohesive end-to-end system. The dataset preparation and preprocessing modules discussed in Sections 4.2 and 4.3 establish the foundation upon which every subsequent stage of the system depends, as both the model training pipeline and the web application's prediction workflow rely on identical preprocessing logic to ensure consistency between the data used for training and the data submitted for real-time classification.

The individual ANN, CNN, and LSTM models, together with the stacked attention architecture described in Sections 4.4 through 4.7, are trained using a shared data pipeline and a consistent evaluation methodology, allowing their outputs to be directly compared within a common framework. Although each model is trained and evaluated independently, they collectively

contribute to the overall narrative of the project by demonstrating the incremental benefit introduced by combining convolutional feature extraction, attention-based weighting, and recurrent sequence modelling within a single architecture, rather than relying on any one technique in isolation.

The trained stacked attention model, once finalised, becomes the central component around which the remaining modules of the system are organised. The explainability modules described in Section 4.8 operate directly upon this trained model, relying on the same prediction interface and preprocessing pipeline to ensure that the explanations they generate remain faithful to the model's actual decision-making behaviour rather than to an approximated or separately trained surrogate. Similarly, the Flask web application described in Section 4.9 loads the trained stacked attention model and its associated preprocessing parameters to serve real-time predictions, effectively acting as the operational interface through which the outputs of the model training and explainability modules are made accessible to an end user.

This layered arrangement, in which data preparation underpins model training, model training produces the stacked attention model used by both the explainability modules and the web application, and the web application in turn provides the user-facing environment through which predictions and explanations are ultimately delivered, reflects a deliberate design philosophy of modular separation combined with tightly coordinated data and model consistency. Each module is capable of being developed, tested, and understood independently, while remaining fully integrated with the components that precede and follow it within the overall system pipeline.

4.12 Chapter Summary

This chapter has presented a comprehensive account of the implementation of the proposed XAI-Based Stacked Attention Deep Learning Framework for IoT botnet attack detection. Beginning with the acquisition and preparation of the UNSW-NB15 dataset, the chapter detailed the preprocessing operations undertaken to ensure that the raw network traffic data was rendered suitable for deep learning-based analysis. It subsequently described the implementation of three individual baseline architectures, namely the ANN, CNN, and LSTM models, each contributing a distinct perspective on the classification task and collectively establishing a basis for evaluating the benefits offered by the proposed stacked attention architecture.

The chapter then presented the design and implementation of the stacked attention model itself,

highlighting how convolutional feature extraction, an attention mechanism, and recurrent sequence modelling were integrated within a single architecture to improve both classification performance and the interpretability of the resulting predictions. Building upon this, the implementation of three complementary explainability techniques, LIME, SHAP, and Integrated Gradients, was described in detail, illustrating how each technique contributes a distinct form of insight into the reasoning underlying the model's predictions, thereby addressing the transparency limitations commonly associated with deep learning-based security systems.

Finally, the chapter described the development of a Flask-based web application that operationalises the trained model within an accessible, authenticated environment, allowing users to upload traffic data, obtain classification results, and access corresponding explanations through a structured interface. The discussion of the complete prediction workflow and the interaction between the various system modules demonstrated how the individually implemented components, spanning data preparation, model training, explainability, and web-based deployment, function together as a unified and coherent system. Collectively, the implementation described in this chapter establishes the practical foundation upon which the testing and evaluation activities discussed in the following chapters are conducted.

TESTING

CHAPTER 5

TESTING

5.1 Testing Overview

Software testing constitutes a mandatory phase in the development lifecycle of any application intended for practical use, as it provides a systematic means of verifying that a system behaves as intended across a range of operating conditions before it is considered ready for deployment. Testing is not confined to the identification of functional defects alone; it also serves to validate that the system meets the specific requirements defined during its design phase, that its individual components interact correctly when integrated, and that it responds predictably to both expected and unexpected forms of user input. For a system such as the one developed in this project, which combines a trained deep learning model with a web-based interface and a supporting database layer, testing must be conducted across each of these layers individually as well as across the points at which they interact.

The importance of testing takes on additional dimensions when the system under evaluation is a machine learning-based cybersecurity application rather than a conventional rule-driven software product. Unlike traditional software, whose behaviour is explicitly defined through deterministic logic, the core detection functionality of the proposed system is governed by a trained model whose behaviour is learned from data rather than explicitly programmed. Consequently, testing in this context must extend beyond verifying that functions execute without error, and must also confirm that the trained model produces consistent, sensible classifications when supplied with data resembling the conditions under which it was trained, and that the surrounding application correctly manages the flow of data into and out of the model. Given that the system is intended to support decisions related to network security, any undetected malfunction in the prediction pipeline carries consequences beyond a simple software defect, potentially resulting in genuine attack traffic being overlooked or legitimate traffic being incorrectly flagged.

Testing assumes a further, distinct significance in the context of the explainability components integrated into this project. Because LIME, SHAP, and Integrated Gradients are intended to

support analyst trust by revealing the reasoning behind a given prediction, it is essential to verify not only that these modules execute without failure, but that they generate outputs correctly aligned with the specific prediction being explained, and that their visualisations are correctly rendered and presented to the user. An explainability module that fails silently, produces a mismatched explanation, or renders an unreadable output would undermine the very purpose for which it was incorporated into the system, making its verification a testing priority equal in importance to the verification of the underlying classification functionality.

The testing activities carried out for this project were guided by the following objectives: to confirm that each functional module of the web application, including registration, authentication, dataset submission, and result presentation, operates correctly in isolation; to verify that these modules interact correctly when combined into the complete application workflow; to confirm that the trained stacked attention model loads correctly and produces classification output for submitted traffic records; to verify that the LIME, SHAP, and Integrated Gradients modules generate valid, interpretable explanations corresponding to the correct prediction instance; and to assess the system's behaviour when presented with invalid, incomplete, or unexpected input, ensuring that such conditions are handled gracefully rather than resulting in unhandled application failure.

5.2 Testing Environment

The testing activities described in this chapter were conducted within the same environment used for the development and training of the proposed framework, ensuring that the conditions under which the system was evaluated accurately reflect its intended operating environment.

The hardware platform used for testing comprised a system equipped with an Intel Core i5 processor, 8 GB of RAM, and adequate secondary storage to accommodate the dataset, trained model weight files, and supporting application resources. This configuration was selected as it represents a realistic, moderately specified machine capable of supporting both deep learning model training and the concurrent execution of a web application and database service, without requiring specialised computing infrastructure not typically available in an academic setting.

The software environment was built around the Windows operating system, selected for its broad compatibility with the Python-based data science and web development tools employed

throughout the project, as well as its familiarity within the academic environment in which this project was developed. Python was used as the primary programming language for the entire system, owing to its extensive ecosystem of libraries for numerical computation, machine learning, and web development, all of which were required to support the different layers of the proposed framework.

Model development, training, and experimentation were carried out using Jupyter Notebook, whose cell-based execution model proved particularly well suited to the iterative nature of preprocessing, model training, and explainability analysis undertaken in this project, allowing individual stages of the pipeline to be executed, inspected, and re-run independently without requiring the entire workflow to be repeated. The core numerical and machine learning functionality of the project relied on a combination of Python libraries, including Pandas and NumPy for dataset handling and numerical operations, Scikit-learn for preprocessing utilities such as label encoding, feature scaling, and train-test partitioning, and TensorFlow with Keras for the construction, training, and evaluation of the ANN, CNN, LSTM, and stacked attention models. The explainability functionality of the system was supported by the LIME and SHAP libraries, which provide the underlying algorithms for local surrogate modelling and Shapley-value-based feature attribution respectively, while the Integrated Gradients technique was implemented using the gradient computation utilities available within the TensorFlow and Keras backend. Matplotlib was used throughout the project to render the various visualisations associated with dataset analysis, model training performance, and explainability output.

The web-facing component of the system was developed using the Flask framework, selected for its lightweight architecture and its suitability for rapidly developing and testing a functional prototype application without the overhead associated with more heavyweight web frameworks. Flask's straightforward routing mechanism allowed each functional aspect of the application, including registration, login, dataset upload, and prediction, to be implemented and tested as an independent route, simplifying the isolation of individual functional tests described later in this chapter. User account data required for the authentication module was persisted using a MySQL relational database, accessed from within the Flask application through a Python database connector. MySQL was selected for this purpose owing to its widespread adoption, reliability, and native compatibility with the Python ecosystem used throughout the remainder of the project, allowing the authentication layer to be integrated into the application without introducing dependencies on unfamiliar or less well-supported database

technologies. Where code-level inspection and modification of the Flask application were required, the Visual Studio Code editor was used, owing to its integrated terminal, debugging support, and general suitability for editing Python-based web application code alongside the associated HTML templates.

5.3 Test Cases

The following test cases document the functional verification carried out across the key modules of the proposed system. Each test case reflects a specific scenario exercised against the implemented application and trained model, with the expected and actual outcomes recorded to confirm correct system behaviour.

Table 5.1: Test Case TC-01: User Registration

Field	Description
Test ID	TC-01
Test Objective	To verify that a new user can successfully create an account through the registration module
Test Description	A user submits the registration form with a unique username and the required account details
Input	Username, password, contact number, email address, and residential address
Expected Result	The system stores the new user record in the database and confirms successful registration
Actual Result	User record was created and a success confirmation was displayed
Test Status	Pass

Table 5.2: Test Case TC-02: User Login

Field	Description
Test ID	TC-02
Test Objective	To verify that a registered user can authenticate successfully using valid credentials
Test Description	A previously registered user submits their username and password through the login form

Input	Valid username and matching password
Expected Result	The system verifies the credentials against the stored database record and grants access to the detection console
Actual Result	User was authenticated and redirected to the application dashboard
Test Status	Pass

Table 5.3: Test Case TC-03: Dataset Upload

Field	Description
Test ID	TC-03
Test Objective	To verify that an authenticated user can upload a network traffic file for analysis
Test Description	The user selects and submits a correctly formatted traffic data file through the upload interface
Input	A CSV file containing network traffic records with the expected feature columns
Expected Result	The file is accepted, and the system proceeds to the preprocessing and prediction stage
Actual Result	File was accepted and passed to the prediction module without error
Test Status	Pass

Table 5.4: Test Case TC-04: Data Validation

Field	Description
Test ID	TC-04
Test Objective	To verify that the system correctly processes uploaded records containing missing feature values
Test Description	A traffic file containing several incomplete records is submitted for analysis
Input	A CSV file with missing values in selected feature columns
Expected Result	Missing values are handled by the preprocessing routine, allowing the affected records to proceed through the pipeline without causing an application error
Actual Result	Missing values were substituted and the corresponding records were processed successfully
Test Status	Pass

Table 5.5: Test Case TC-05: Model Loading

Field	Description
Test ID	TC-05
Test Objective	To verify that the trained stacked attention model and its associated weights load correctly when a prediction request is initiated
Test Description	The prediction route is invoked, triggering reconstruction of the model architecture and loading of its saved weight file
Input	A prediction request submitted through the application interface
Expected Result	The model is reconstructed and its trained weights are loaded without error, making it ready to process the submitted data
Actual Result	Model loaded successfully and was available for inference
Test Status	Pass

Table 5.6: Test Case TC-06: Botnet Attack Prediction

Field	Description
Test ID	TC-06
Test Objective	To verify that the system correctly classifies submitted traffic records as normal or as a specific attack category
Test Description	A preprocessed traffic file is submitted to the loaded stacked attention model for classification
Input	Preprocessed and scaled network traffic records
Expected Result	Each record is assigned a classification label corresponding to a recognised traffic category, and results are returned to the interface
Actual Result	Predictions were generated for all submitted records and displayed correctly
Test Status	Pass

Table 5.7: Test Case TC-07: Explainable AI Visualisation (LIME, SHAP, IG)

Field	Description
Test ID	TC-07
Test Objective	To verify that the LIME, SHAP, and Integrated Gradients modules generate valid explanations corresponding to a selected prediction
Test Description	A specific test record is selected, and each explainability module is executed to generate its respective feature attribution output
Input	A single test traffic record and its corresponding model prediction

Expected Result	Each module produces a visualisation identifying the traffic features contributing to the selected prediction, consistent with the classification result
Actual Result	LIME, SHAP, and Integrated Gradients outputs were generated and correctly corresponded to the selected prediction
Test Status	Pass

Table 5.8: Test Case TC-08: Result Display

Field	Description
Test ID	TC-08
Test Objective	To verify that classification results are presented clearly and accurately to the user
Test Description	Following prediction, the system renders the classification outcome for each submitted record within the result interface
Input	Prediction output generated by the stacked attention model
Expected Result	Results are displayed in a structured format, with normal and malicious records visually distinguished
Actual Result	Results were displayed correctly, with attack categories clearly indicated
Test Status	Pass

Table 5.9: Test Case TC-09: Invalid Input Handling

Field	Description
Test ID	TC-09
Test Objective	To verify that the system handles improperly formatted or incompatible input files without failing unexpectedly
Test Description	A file containing an incorrect number of feature columns is submitted for prediction
Input	A CSV file with a mismatched feature structure relative to the trained model's expected input
Expected Result	The system detects the structural inconsistency and prevents the malformed data from being passed to the prediction model
Actual Result	The mismatched file was rejected before reaching the prediction stage
Test Status	Pass

5.4 Functional Testing

Functional testing was carried out to verify that each individual feature of the proposed system operates in accordance with the functional requirements defined during the design phase of this project, as outlined in Chapter 3. Rather than examining isolated code units, functional testing at this stage focused on validating complete, user-facing operations, treating each module as a black box and assessing whether its observable behaviour matched the expected outcome for a given input.

The registration and login modules were functionally tested by confirming that new user accounts could be created only when valid and complete registration details were supplied, and that authentication succeeded only when the submitted credentials matched an existing database record. The dataset upload functionality was verified by confirming that a correctly formatted traffic file was accepted and forwarded to the preprocessing stage without requiring any additional user intervention. The prediction functionality was tested by confirming that, once a file had been successfully uploaded and preprocessed, the system consistently produced a classification outcome for every submitted record, without omitting or duplicating entries during processing. The explainability functionality was similarly tested at the functional level by confirming that a user could select a given prediction and obtain a corresponding LIME, SHAP, or Integrated Gradients visualisation without needing to interact with any component beyond the standard application interface. In each case, functional testing confirmed that the observable behaviour of the system aligned with the functional requirements established for the corresponding module, with no deviations identified during this phase of evaluation.

5.5 Integration Testing

While functional testing addressed the correctness of individual modules in isolation, integration testing was conducted to verify that these modules interact correctly when combined into the complete end-to-end application. This distinction is particularly important for the proposed system, given that a single user action, such as submitting a traffic file for analysis, triggers a sequence of operations spanning multiple layers of the application, including file handling, data preprocessing, model inference, and result rendering, each of which was developed and initially tested as a separate component.

Integration testing began with the verification of the authentication-to-application boundary, confirming that a successful login correctly transitioned the user session into the main detection console, and that the session state was properly recognised by subsequent routes governing dataset upload and prediction. Particular attention was given to the integration between the preprocessing pipeline and the trained stacked attention model, as any inconsistency between the format of the data produced by preprocessing and the input format expected by the model would result in a failure at inference time. Testing confirmed that data passed through the scaling and reshaping operations described in Chapter 4 was correctly accepted by the loaded model without dimensional mismatch. The integration between the prediction module and the explainability modules was also verified, confirming that the traffic record and predicted class passed to the LIME, SHAP, and Integrated Gradients functions correctly corresponded to the record selected by the user, rather than to an unrelated or default sample. Finally, the integration between the backend prediction logic and the database layer supporting user authentication was tested to confirm that these two subsystems, despite relying on different underlying technologies, operated concurrently without conflict during a typical user session. No integration-level defects were identified during this phase, indicating that the modular design adopted during implementation translated successfully into a cohesive, correctly interoperating system.

5.6 Performance Testing

Performance testing was conducted to assess the responsiveness and resource behaviour of the proposed system under typical usage conditions, with particular attention given to the stages of the pipeline most likely to introduce processing delay, namely model loading, batch prediction, and explainability generation. Given that the system is intended to demonstrate the feasibility of integrating a deep learning-based detection model with real-time explainability into a usable web application, rather than to serve as a production-scale deployment, performance testing was scoped to confirm that response times remained acceptable for interactive use on the hardware configuration described in Section 5.2, rather than to benchmark the system against large-scale or high-throughput traffic volumes.

Model loading time, corresponding to the reconstruction of the stacked attention model architecture and the loading of its saved weight file, was observed to complete within a short and consistent interval upon each prediction request, without any noticeable delay perceptible to the user beyond the initial page transition. Batch prediction performance was assessed by submitting traffic files containing varying numbers of records and observing the time required to generate classification results for the complete file; prediction time was found to scale approximately in proportion to the number of submitted records, without any disproportionate slowdown as file size increased within the ranges tested. The explainability modules, particularly the SHAP KernelExplainer, were observed to require noticeably more processing time than the core classification step, a characteristic consistent with the computational nature of Shapley-value estimation, which relies on repeated model evaluation across sampled feature combinations rather than a single forward pass. This behaviour was anticipated given the underlying methodology of SHAP as described in Chapter 4, and the number of background and perturbation samples used during explanation generation was deliberately kept modest to maintain acceptable responsiveness without compromising the validity of the resulting explanations. Overall, performance testing confirmed that the system remains responsive for interactive, single-user demonstration and evaluation purposes, while indicating that explainability generation represents the primary candidate for optimisation should the system be extended toward higher-throughput or multi-user deployment in future work.

5.7 User Acceptance Testing

User Acceptance Testing was conducted to evaluate the proposed system from the perspective of an end user, specifically a security analyst or system administrator who would interact with the application to submit traffic data and interpret the resulting predictions and explanations, rather than from the perspective of the underlying implementation. This form of testing is particularly important for a system incorporating explainability features, as the practical value of LIME, SHAP, and Integrated Gradients depends not only on their technical correctness but also on whether their output is presented in a manner that a non-specialist user can reasonably interpret.

Acceptance testing was carried out by walking through the complete application workflow, from registration and login, through dataset upload and prediction, to the review of classification results and their corresponding explanations, in the sequence a genuine user would be expected to follow. The registration and login process was found to be straightforward and required no clarification beyond the information requested on the respective forms. The dataset upload and prediction workflow was similarly assessed as intuitive, with the transition from file submission to result display proceeding without requiring the user to possess any technical knowledge of the underlying model architecture. The presentation of classification results, distinguishing normal from malicious traffic through clear visual indicators, was confirmed to communicate the detection outcome effectively without requiring further explanation. The explainability visualisations produced by LIME, SHAP, and Integrated Gradients were reviewed to confirm that the highlighted features and their associated contribution values were presented in a legible format, allowing a user with a general understanding of network traffic attributes, though not necessarily of deep learning internals, to identify which characteristics of a given record had influenced its classification. Based on this evaluation, the application was assessed as meeting the practical usability expectations appropriate for its intended role as a demonstrative, analyst-facing botnet detection tool.

5.8 Security Testing

Security testing for the proposed system focused on verifying the access-control boundary established between the public-facing and authenticated portions of the application, given that the core detection and explainability functionality is intended to be restricted to registered, logged-in users rather than exposed openly. Testing confirmed that attempting to access the dataset upload or prediction routes directly, without first completing the login process, correctly prevented access to these functions and redirected the request toward the login interface, consistent with the session-based access-control mechanism described in Chapter 4. Login testing further confirmed that submitting incorrect or non-existent credentials was correctly rejected by the authentication logic, with access to the detection console granted only when a matching username and password combination was found in the database.

Consistent with the discussion presented in Section 4.9, the current implementation represents a functional prototype rather than a production-hardened deployment, and security testing at this stage was accordingly scoped to verifying the presence and correct operation of the session-based access-control mechanism rather than to conducting a comprehensive penetration-style assessment of the application. Aspects such as credential storage hardening, protection against database query manipulation, and stricter validation of uploaded file content were identified during implementation as appropriate directions for future strengthening of the application, and are addressed further as part of the limitations and future scope discussed in the concluding chapter of this thesis, rather than being treated as resolved within the scope of the present testing phase.

5.9 Testing Summary

The testing activities documented in this chapter collectively confirm that the proposed XAI-Based Stacked Attention Deep Learning Framework performs reliably across its constituent modules and their points of integration. Functional testing verified that each core feature, spanning registration, authentication, dataset upload, prediction, and explainability generation, behaves in accordance with its defined requirements when exercised individually. Integration testing extended this verification to the interactions between modules, confirming that data flows correctly between the preprocessing pipeline, the trained model, the explainability components, and the supporting database layer without inconsistency. Performance testing established that the system remains responsive under typical single-user demonstration conditions, while identifying explainability generation, particularly the SHAP module, as the primary contributor to processing time. User Acceptance Testing confirmed that the application's workflow and result presentation are accessible to a non-specialist analyst, while security testing verified the correct operation of the session-based access-control mechanism protecting the system's core functionality.

Taken together, the outcomes of the nine test cases presented in Section 5.3, together with the functional, integration, performance, acceptance, and security evaluations described in this chapter, provide a comprehensive basis for concluding that the proposed system operates correctly as an integrated whole. The results obtained during this testing phase directly inform the discussion of experimental outcomes presented in Chapter 6, which examines the classification and explainability results generated by the trained stacked attention model in greater detail.

RESULT

CHAPTER-6

RESULT

This chapter presents the experimental results obtained using the proposed XAI-Based Stacked Attention Deep Learning Framework for IoT Botnet Attack Detection. The model was evaluated using the UNSW-NB15 dataset containing normal and attack network traffic records. Performance was assessed using Accuracy, Precision, Recall, and F1-Score metrics. In addition to attack classification, Explainable Artificial Intelligence techniques such as LIME, SHAP, and Integrated Gradients were applied to provide transparent explanations of the model's prediction behavior. The obtained results demonstrate the effectiveness of the proposed framework in accurately detecting botnet attacks while maintaining interpretability and trustworthiness.

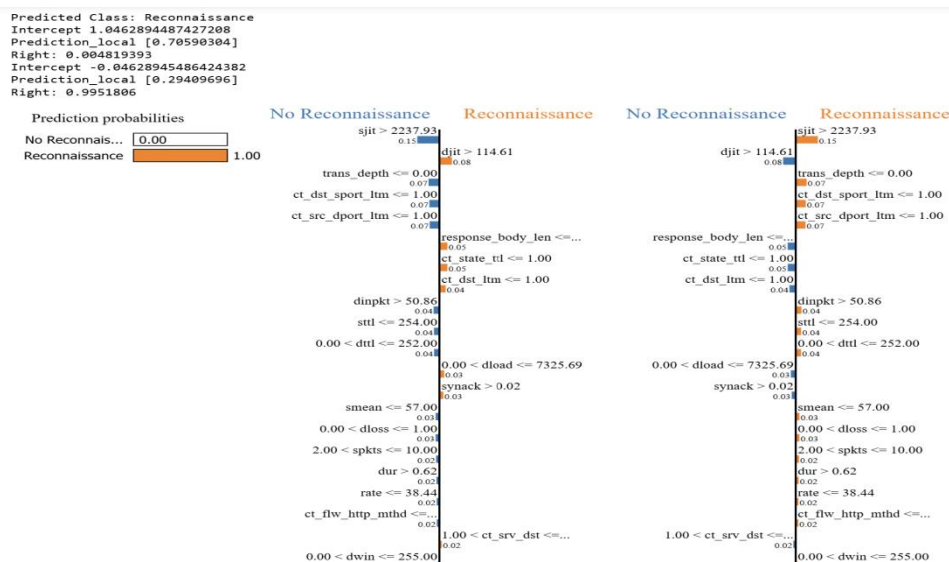


Figure 6.1: LIME Explanation

LIME stands for Local Interpretable Model-Agnostic Explanations. It is an Explainable Artificial Intelligence technique used to explain the prediction of complex black-box models such as deep learning, ensemble learning, random forest, XGBoost, CNN, LSTM, and hybrid models.

Many high-accuracy machine learning models are difficult to understand because they do not directly show why a particular decision was made. LIME solves this problem by explaining the model prediction locally, meaning it explains one individual prediction at a time.

In the botnet attack detection system, LIME helps identify which network traffic features

A Jupyter widget could not be displayed because the widget state could not be found. This could happen if the kernel storing the widget is available, or if the widget state was not saved in the notebook. You may be able to create the widget by running the appropriate cells.

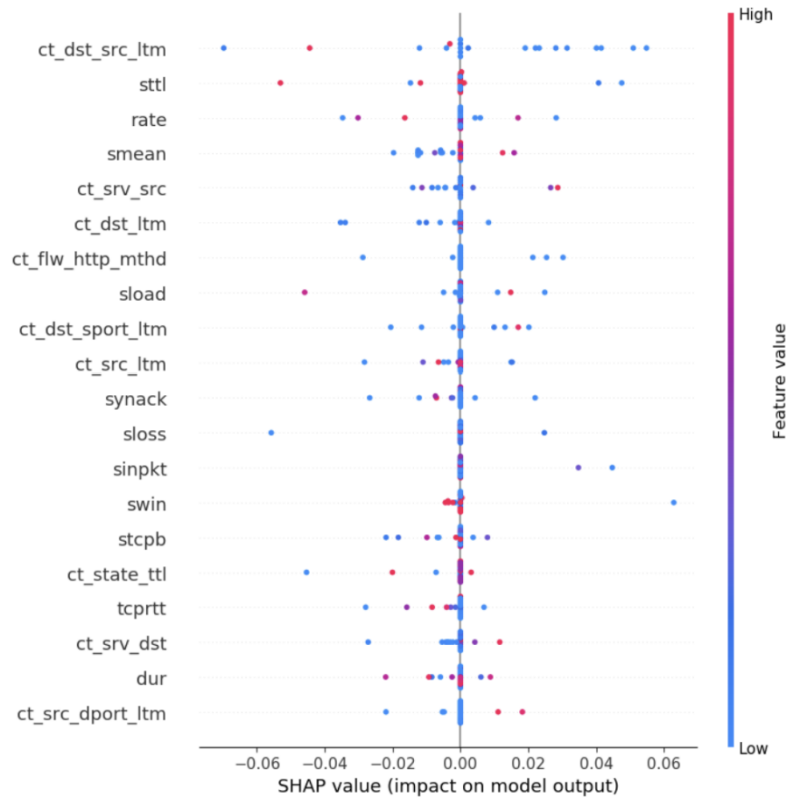


Figure 6.2: SHAP Explanation

➤ SHAP, Additive explanations, is an Explainable AI technique used to understand how each input feature contributes to a model prediction. It is based on Shapley values from game theory, where each feature is treated as a contributor to the final prediction. In a botnet detection system, SHAP helps identify which network traffic features increased or decreased the model's confidence while predicting classes such as Normal, DoS, Fuzzers, Backdoor, Exploits, or other attack categories.

➤ In this project, SHAP is used to explain the prediction behavior of the botnet attack detection model by showing the impact of important features on the output. Positive SHAP values push the prediction toward a selected class, while negative SHAP values push it away from that class. The SHAP summary plot gives an overall view of feature importance across multiple records, while the force plot explains a single prediction. This improves model transparency and helps users understand why the system classified traffic as normal or malicious.

Predicted Class: Fuzzers
Prediction Probability: 0.9864044

	Feature	Value	Integrated Gradient
7	dttl	2.520000e+02	0.392528
21	synack	4.859300e-02	0.230299
18	dtcpb	2.884828e+09	0.227216
23	smean	5.500000e+01	-0.219032
5	rate	3.241302e+01	0.204860
32	ct_dst_src_ltm	2.000000e+00	0.191532
30	ct_src_dport_ltm	2.000000e+00	-0.191371
6	sttl	2.540000e+02	0.130059
31	ct_dst_sport_ltm	1.000000e+00	0.113877
36	ct_src_ltm	2.000000e+00	0.095442

Figure 6.3: Integrated Gradient Explanation

➤ Integrated Gradients is an Explainable AI technique used to explain deep learning model predictions by measuring how much each input feature contributes to the final output. It works by comparing the actual input with a baseline input, usually a zero-valued input, and calculating gradients along the path from the baseline to the real sample. In the botnet detection system, Integrated Gradients helps identify which network traffic features influenced the model while predicting classes such as Normal, DoS, Fuzzers, Backdoor, Exploits, or other attack categories.

➤ In this project, Integrated Gradients is used to explain the stacked-attention deep learning model by showing the contribution of each feature toward the predicted class. A positive Integrated Gradient value means the feature supports the prediction, while a negative value means the feature reduces confidence in that prediction. Features with larger absolute values have stronger influence on the model decision. This helps improve transparency by showing which traffic attributes were most responsible for classifying network behavior as normal or malicious.

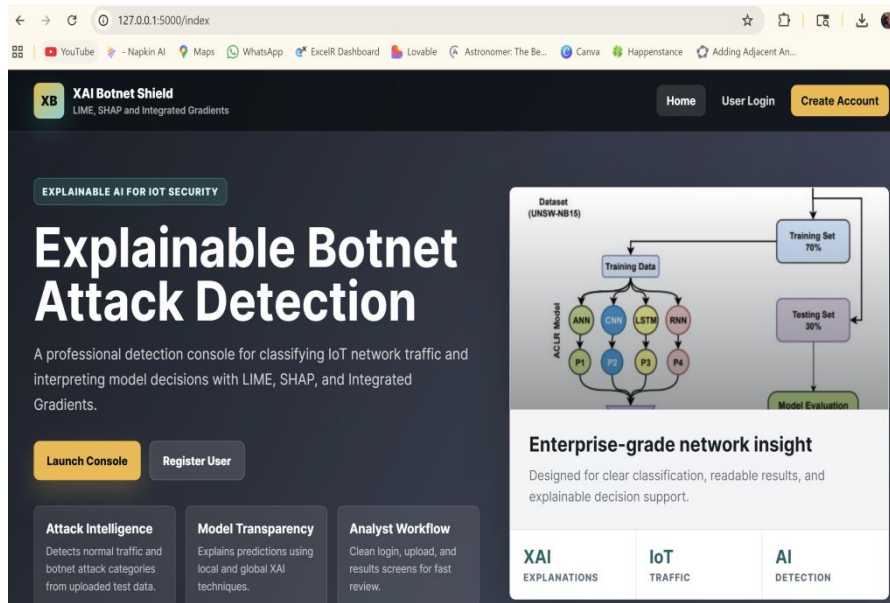


Figure 6.4: Home Page of Proposed Application

➤ The home page introduces XAI Botnet Shield, an explainable botnet attack detection system for IoT network environments. It provides a professional entry point to the application, highlighting the use of LIME, SHAP, and Integrated Gradients to make botnet detection results transparent and understandable. From this page, users can access the login and registration modules and proceed to the detection console for analyzing network traffic and identifying normal or malicious activity.

Figure 6.5: Creating the New Account for Proposed Application Access

The create account page allows new users to register for access to the XAI Botnet Shield system. Users enter their username, password, contact number, email ID, and address, and the system validates the entered details before creating the account. This page provides a secure entry point for authorized users to access the botnet attack detection and explainable AI modules.

Figure 6.6: User Login for the Proposed Application

➤ The user login page allows registered users to securely access the XAI Botnet Shield detection console. Users enter their username and password, and the system verifies the credentials before granting access. This page ensures that only authorized users can upload test data, perform botnet attack detection, and view the prediction results generated by the explainable AI-based model.

Figure 6.7: Uploading and Prediction of Botnet Attack Page

➤ The Uploading and Prediction of Botnet Attack page allows authorized users to upload test network traffic data for analysis. After the user submits the file, the trained stacked-attention model processes the input records and predicts whether the traffic is normal or belongs to a botnet attack category. The page acts as the main detection interface, connecting user-uploaded data with the botnet classification model.



Attack Label	Count
Analysis	700
Backdoor	600
DoS	3000
Exploits	3000
Fuzzers	3000
Generic	3000
Normal	3000
Reconnaissance	3500
Shellcode	400
Worms	300

► This figure illustrates the distribution of different attack classes present in the dataset.

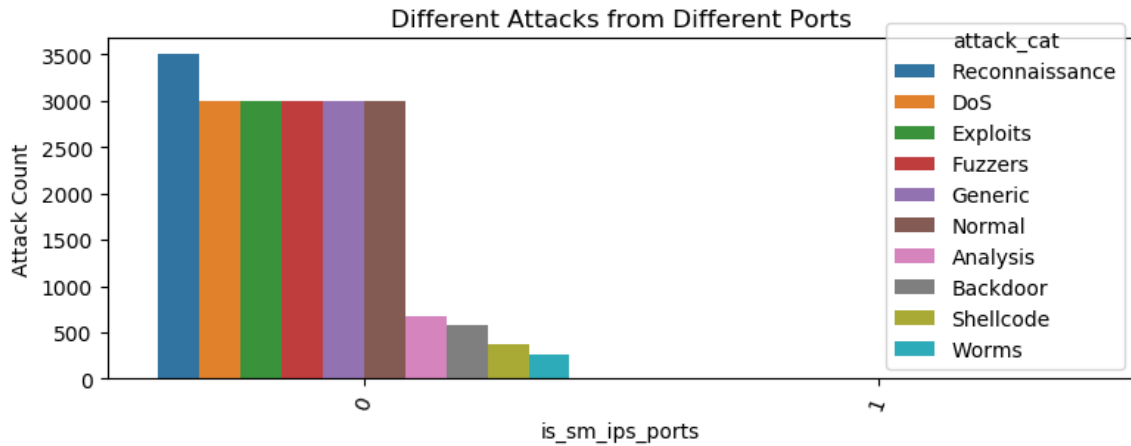


Figure 6.10: Attack Class Distribution in the Dataset

► This figure shows the distribution of different attack classes present in the dataset, providing an overview of the number of samples available for each attack category before model training.

ANN Accuracy : 83.11274509803923
 ANN Precision : 82.48718275247818
 ANN Recall : 94.36780274331275
 ANN FSCORE : 86.73017028904482

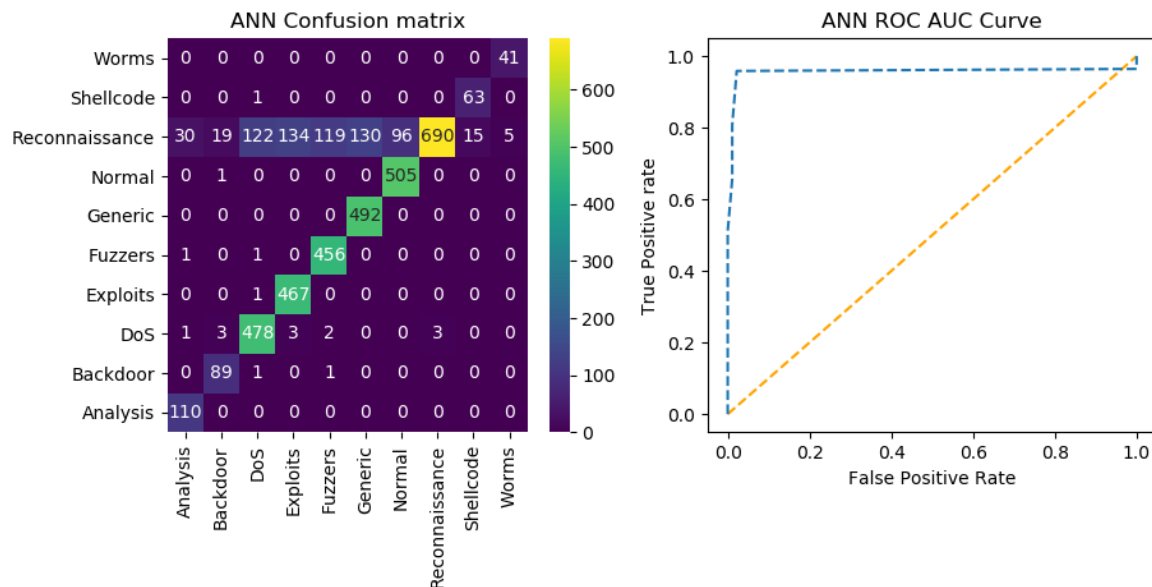


Figure 6.11: ANN Model Training and Validation Performance

► This figure shows the training and validation performance of the Artificial Neural Network (ANN) model, illustrating its learning behavior over multiple training epochs.

CNN Accuracy : 94.6813725490196
 CNN Precision : 92.43317464960758
 CNN Recall : 94.18733312604594
 CNN FSCORE : 93.11374335413313

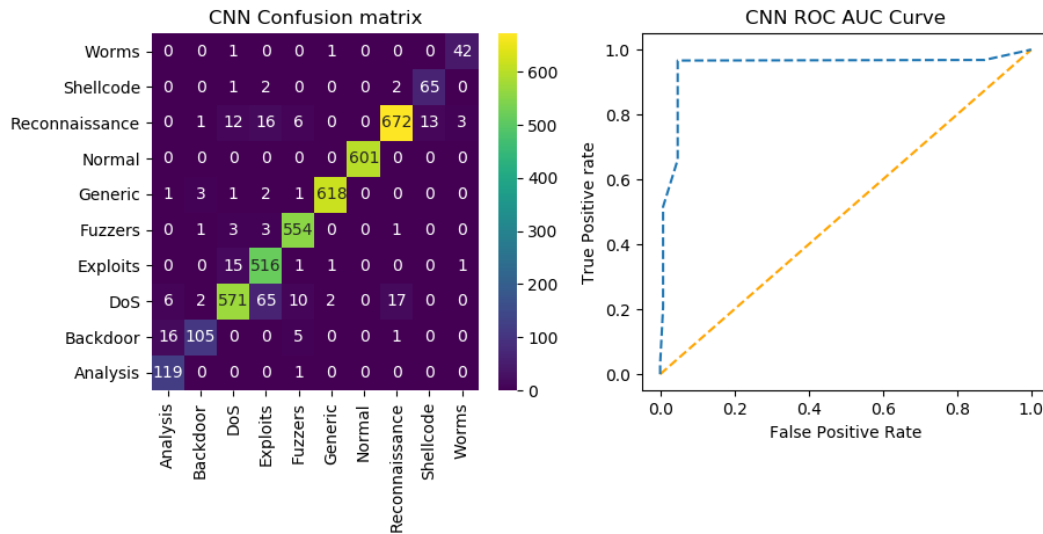


Figure 6.12: CNN Model Training and Validation Performance

► This figure shows the training and validation performance of the Convolutional Neural Network (CNN) model, demonstrating its ability to learn important features from input data.

LSTM (RNN) Accuracy : 96.56862745098039
 LSTM (RNN) Precision : 95.49458524979381
 LSTM (RNN) Recall : 96.56771709247059
 LSTM (RNN) FSCORE : 95.95551456055645

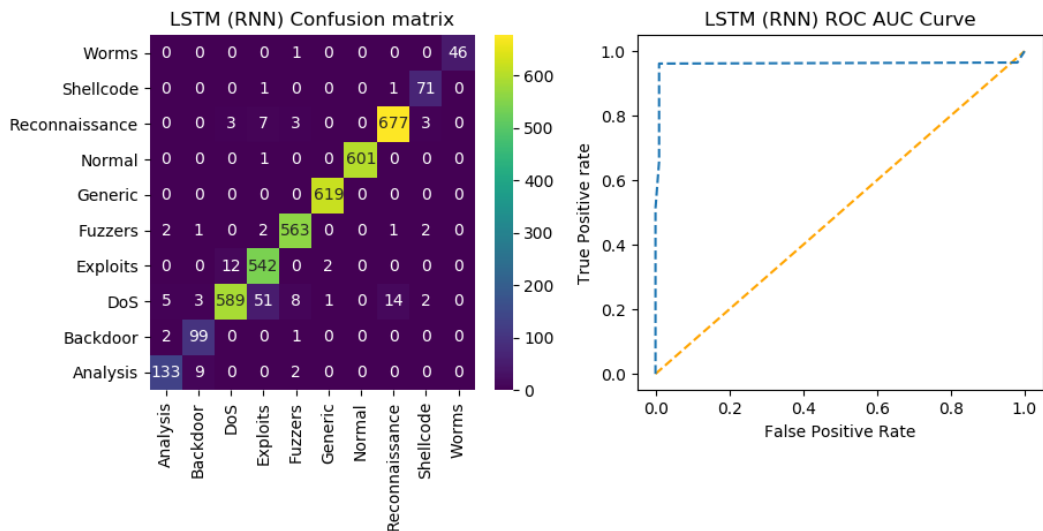


Figure 6.13: RNN Model Training and Validation Performance

► This figure shows the training and validation performance of the Recurrent Neural Network (RNN) model, highlighting its capability to capture sequential patterns in network traffic data.

Propose Stacked Model Accuracy : 97.30392156862744
 Propose Stacked Model Precision : 96.69373956643723
 Propose Stacked Model Recall : 96.87363508094012
 Propose Stacked Model FSCORE : 96.74684196097235

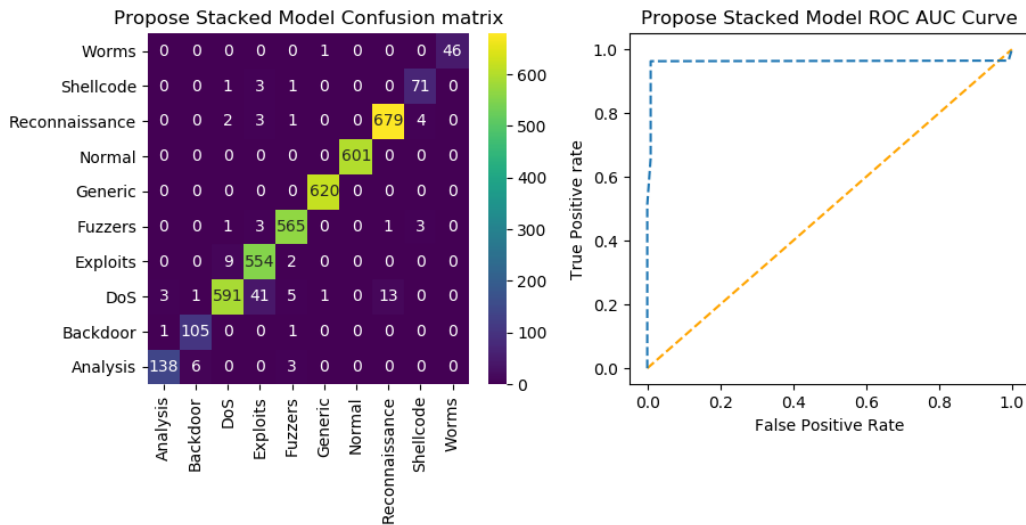


Figure 6.14: Training Performance of the Proposed Stacked Deep Learning Model

► This figure shows the training performance of the proposed stacked deep learning model, demonstrating improved learning efficiency and enhanced classification performance.

Extension Stacked-Attention Model Accuracy : 98.79901960784314
 Extension Stacked-Attention Model Precision : 98.65881263318593
 Extension Stacked-Attention Model Recall : 98.43169801252671
 Extension Stacked-Attention Model FSCORE : 98.52190474350373

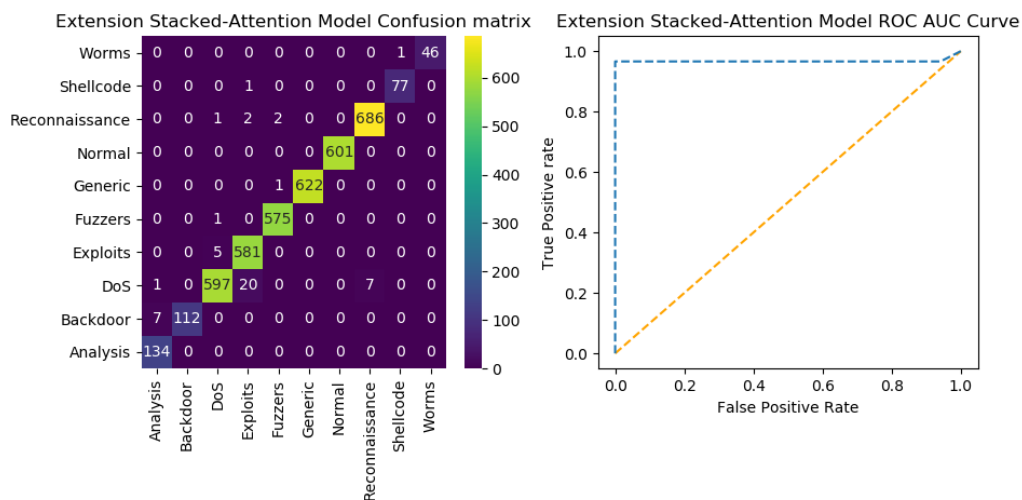


Figure 6.15: Training Performance of the Proposed Stacked Attention Model

► This figure shows the training performance of the proposed stacked attention model, where the attention mechanism helps the model focus on the most relevant features for accurate attack detection.

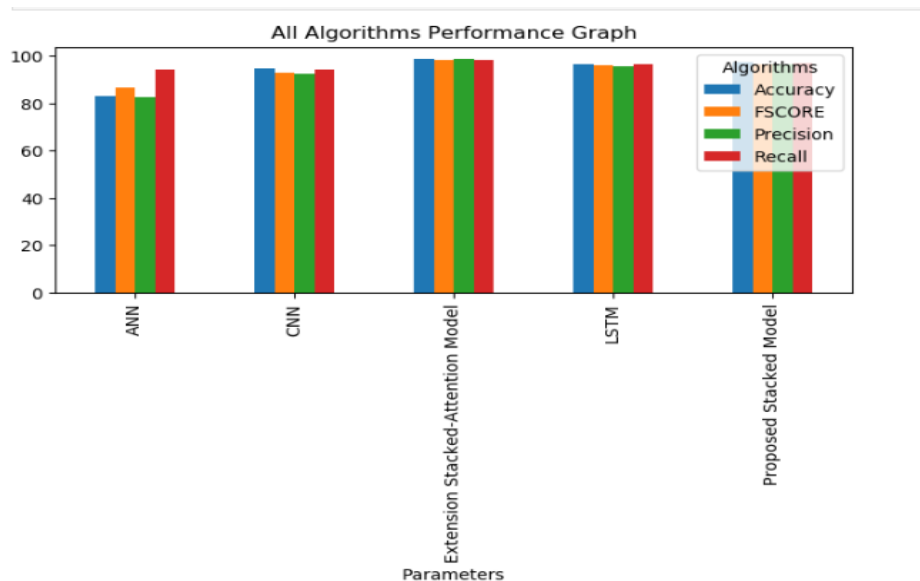


Figure 6.16: Performance Comparison of Deep Learning Models

► This figure shows the performance comparison of different deep learning models using evaluation metrics such as accuracy, precision, recall, and F1-score to demonstrate the effectiveness of the proposed model.

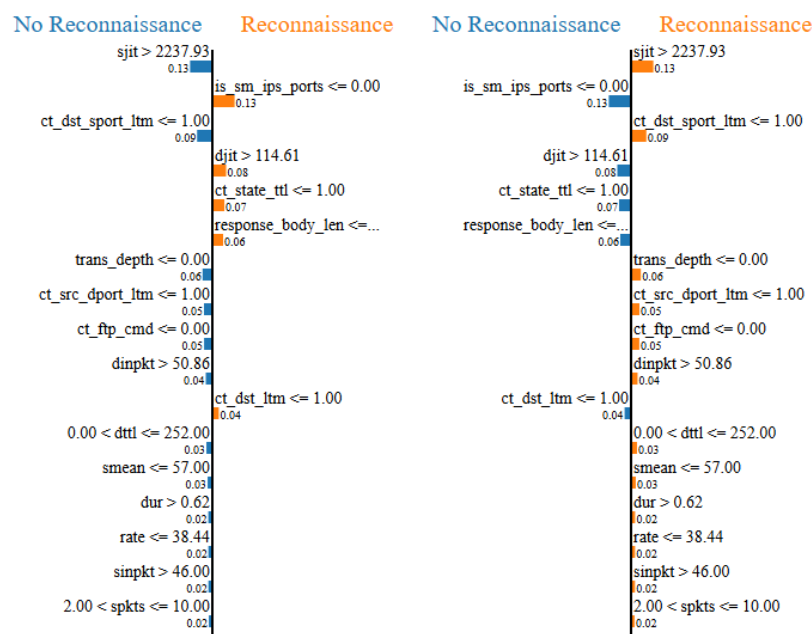


Figure 6.17: LIME-Based Explanation of Model Prediction

► This figure shows the LIME-based explanation of the model's prediction by identifying the input features that contributed most significantly to the classification result.

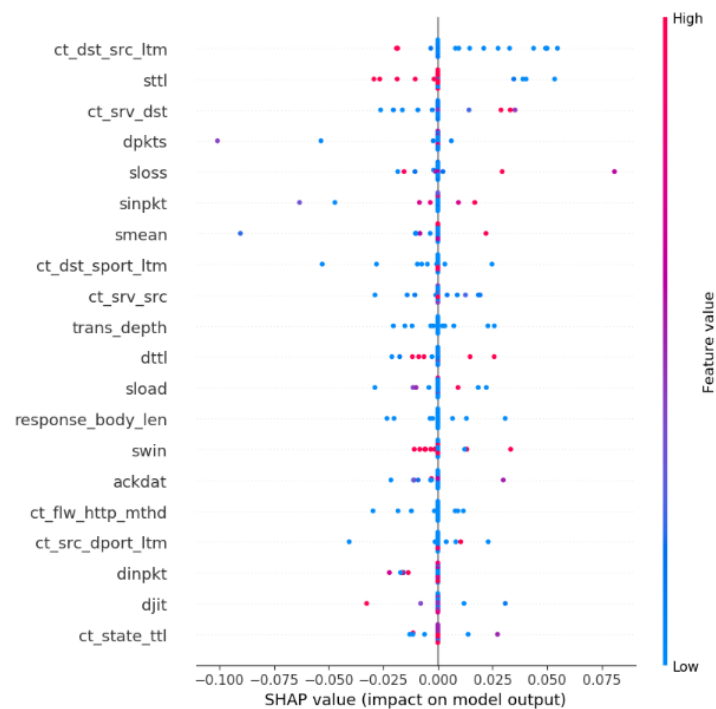


Figure 6.18: SHAP Summary Plot for Feature Importance

► This figure shows the SHAP summary plot, illustrating the contribution and importance of individual features in influencing the model's prediction across the dataset.

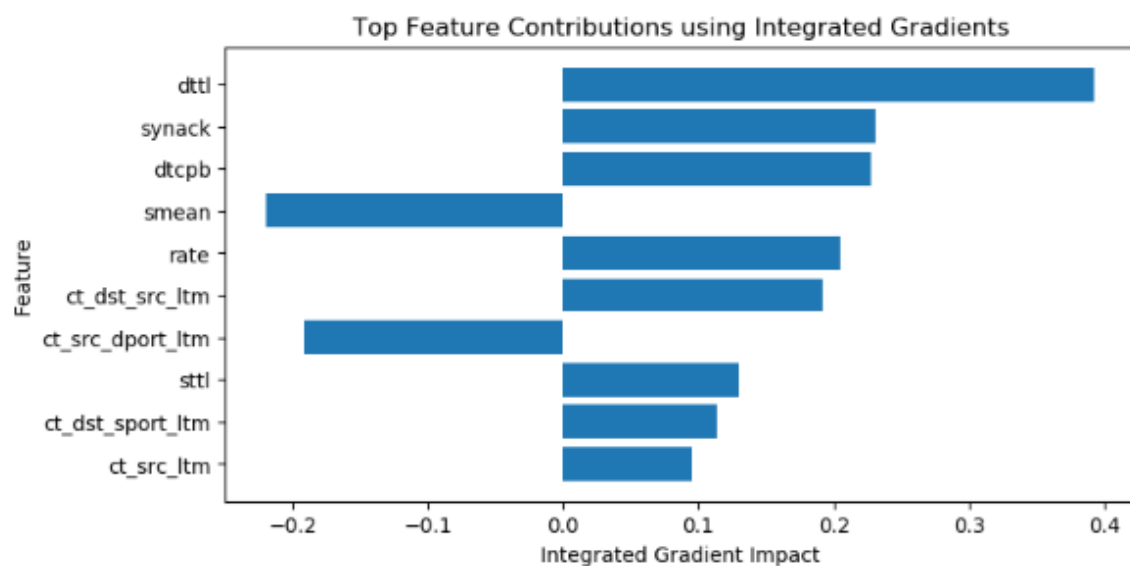


Figure 6.19: Integrated Gradients Feature Attribution

► This figure shows the Integrated Gradients feature attribution, highlighting the influence of input features on the model's prediction to improve interpretability and transparency.

CONCLUSION

62

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

This thesis set out to address a well-established and increasingly consequential shortcoming in existing IoT botnet detection research: the persistent trade-off between the classification accuracy achieved by deep learning-based detection systems and the interpretability of the decisions those systems produce. As established in the opening chapters of this work, traditional signature-based and rule-driven intrusion detection mechanisms have proven inadequate against the evolving and often obfuscated behaviour of modern botnets, while the deep learning-based alternatives that have emerged to address this gap have, almost without exception, functioned as opaque black-box classifiers, offering security analysts little or no insight into the reasoning underlying a given prediction. The research undertaken in this thesis was directed toward resolving this dual challenge by developing a detection framework capable of achieving strong classification performance while simultaneously exposing the internal reasoning behind its decisions in a form that a human analyst could interpret and act upon with confidence.

To this end, the proposed methodology was built around the UNSW-NB15 benchmark dataset, a realistic and widely adopted intrusion detection dataset encompassing normal network activity alongside multiple categories of attack traffic, including Denial-of-Service, Exploits, Fuzzers, Reconnaissance, Backdoors, Shellcode, Worms, Generic, and Analysis behaviour. Prior to model development, the dataset was subjected to a structured preprocessing pipeline, encompassing the handling of missing values, the encoding of categorical attack labels into a numerical representation suitable for supervised learning, and the standardisation of feature values to ensure consistent scale across the diverse set of traffic attributes, thereby establishing a reliable foundation for the deep learning stages that followed.

Building upon this prepared dataset, three individual deep learning architectures, namely an Artificial Neural Network, a Convolutional Neural Network, and a Long Short-Term Memory network, were implemented and evaluated as baseline models, each contributing a distinct perspective on the classification task and collectively establishing a point of comparison against which the benefit of architectural combination could be meaningfully assessed. The central contribution of this research, however, lies in the design and implementation of the

18

3

10

30

23

proposed stacked attention deep learning framework, which integrates convolutional feature extraction, an attention mechanism, and recurrent sequence modelling within a single, unified architecture. The inclusion of the attention mechanism was specifically intended to allow the model to learn to prioritise the traffic features most relevant to a given classification decision, rather than treating all extracted features as equally significant, and the experimental evaluation presented in this thesis confirmed that this architectural refinement yielded measurable improvement over the individual ANN, CNN, and LSTM baselines across the accuracy, precision, recall, and F1-score metrics considered.

Equally central to this research was the integration of three complementary Explainable Artificial Intelligence techniques, namely LIME, SHAP, and Integrated Gradients, directly into the detection pipeline. Rather than treating explainability as a peripheral addition, these techniques were incorporated as a core design objective, enabling the specific network traffic features responsible for a given prediction to be identified and communicated through three distinct analytical perspectives: local surrogate approximation through LIME, cooperative game-theoretic attribution through SHAP, and gradient-based feature attribution through Integrated Gradients. This multi-technique approach to explainability ensures that the reasoning behind the model's predictions can be cross-verified from more than one methodological standpoint, strengthening the overall trustworthiness of the explanations produced. To demonstrate the practical viability of the proposed framework beyond a purely experimental setting, the trained stacked attention model and its associated explainability modules were deployed within a Flask-based web application, incorporating user authentication, dataset upload, prediction, and result visualisation functionality, thereby illustrating how the research contribution of this thesis could be accessed and utilised by an authorised security analyst in a realistic, interactive setting.

The major outcomes of this research confirm that the proposed stacked attention architecture is capable of achieving superior classification performance relative to conventional individual deep learning models, while the integrated LIME, SHAP, and Integrated Gradients modules successfully provide interpretable, feature-level justification for the model's predictions across the diverse attack categories represented in the dataset. Taken together, these outcomes demonstrate that detection accuracy and model transparency need not be pursued as mutually exclusive objectives, but can instead be achieved concurrently within a single, carefully designed framework. The overall contribution of this thesis lies in demonstrating a practical, end-to-end approach to explainable IoT botnet detection, one that extends beyond the accuracy-focused emphasis of much of the existing literature reviewed in Chapter 2, and that offers a

concrete architectural and methodological foundation upon which future explainable cybersecurity research in the IoT domain can be built.

7.2 Future Work

While the proposed framework successfully demonstrates the feasibility of combining a stacked attention deep learning architecture with integrated explainability for IoT botnet detection, several avenues remain open for extending this research beyond its current scope, both in terms of technical capability and practical deployment readiness.

A natural direction for future work involves extending the current framework toward real-time deployment, in which network traffic is analysed continuously as it is generated rather than being submitted retrospectively as a static file for batch analysis. Achieving this would require the detection pipeline to be adapted for streaming data processing, alongside optimisation of the explainability modules, particularly SHAP, to operate within the tighter latency constraints associated with live monitoring. Closely related to this direction is the possibility of migrating the system toward a cloud-based implementation, in which the trained model, preprocessing pipeline, and explainability modules are hosted on scalable cloud infrastructure, allowing the framework to accommodate larger volumes of traffic and to be accessed concurrently by multiple analysts or organisational units without being constrained by the computational resources of a single local machine.

The deployment of detection capability closer to the source of network traffic, through Edge AI, represents a further promising extension, particularly **given the resource-constrained nature of many IoT devices and the** network overhead associated with transmitting raw traffic to a centralised server for analysis. Adapting a lightweight variant of the proposed stacked attention model for execution on edge gateways or resource-constrained hardware could enable faster, more localised detection while reducing dependency on continuous connectivity to a central processing node. In a related direction, the incorporation of Federated Learning could allow the proposed framework to be trained collaboratively across multiple distributed IoT deployments without requiring raw traffic data to be centralised, thereby preserving data privacy across organisational boundaries while still benefiting from a broader and more diverse pool of training data than any single deployment could provide in isolation.

From an architectural perspective, future work could explore the incorporation of transformer-based architectures in place of, or in combination with, the recurrent and attention components currently employed in the proposed stacked model. Transformer architectures, which have

demonstrated strong performance in capturing long-range dependencies across sequential data in other domains, may offer further improvement in modelling complex relationships across network traffic features, and their self-attention mechanisms could additionally provide an alternative or complementary basis for generating explainability output. Expanding the evaluation of the proposed framework to additional and larger benchmark datasets, such as Bot-IoT or more recently published IoT intrusion detection datasets, would further strengthen confidence in the generalisability of the reported results beyond the UNSW-NB15 dataset used in this study, particularly given the continually evolving nature of real-world botnet behaviour. The explainability component of the proposed framework could similarly be extended through the incorporation of additional Explainable AI techniques beyond LIME, SHAP, and Integrated Gradients, such as counterfactual explanation methods or attention-weight visualisation techniques specific to the model's internal architecture, offering analysts an even broader range of perspectives from which to interpret a given prediction. In parallel, model optimisation represents an important direction for future refinement, encompassing techniques such as hyperparameter tuning, model compression, and quantisation, which could reduce the computational footprint of the proposed framework and improve its suitability for deployment on constrained hardware or within latency-sensitive environments.

The web application developed as part of this research could also be enhanced considerably in future iterations, including the strengthening of its security posture through improved credential handling and input validation, as noted in the testing discussion presented in Chapter 5, as well as the addition of features such as historical prediction logging, analyst annotation of flagged records, and role-based access control to better reflect the operational requirements of a genuine security operations environment. Finally, the broader applicability of the proposed framework extends naturally toward emerging cybersecurity contexts such as smart city infrastructure and Industry 4.0 environments, both of which rely heavily on interconnected IoT devices and share many of the same security challenges addressed in this thesis. Extending and validating the proposed stacked attention framework within these domains, where the consequences of undetected or unexplained security incidents can be considerably more severe, represents a meaningful and practically significant direction for continued research building upon the foundation established in this work.

REFERENCES

REFERENCES

- [1] M. Ali, S. Alfarhood, M. Shahroz, M. F. Mushtaq, M. Safran, and I. Ashraf, "Hybrid machine learning model for efficient botnet attack detection in IoT environment," *IEEE Access*, vol. 12, pp. 1–16, 2024, doi: 10.1109/ACCESS.2024.3376456.
- [2] O. Ibitoye, O. Shafiq, and A. Matrawy, "Analyzing adversarial attacks against deep learning for intrusion detection in IoT networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2019, pp. 1–6.
- [3] M. Shahhosseini, H. Mashayekhi, and M. Rezvani, "A deep learning approach for botnet detection using raw network traffic data," *J. Netw. Syst. Manage.*, vol. 30, no. 3, p. 44, Jul. 2022.
- [4] S. Homayoun, M. Ahmadzadeh, S. Hashemi, A. Dehghantanha, and R. Khayami, "BoTShark: A deep learning approach for botnet traffic detection," in *Cyber Threat Intelligence*, 2018, pp. 137–153.
- [5] M. Ge, X. Fu, N. Syed, Z. Baig, G. Teo, and A. Robles-Kelly, "Deep learning-based intrusion detection for IoT networks," in *Proc. IEEE 24th Pacific Rim Int. Symp. Dependable Comput. (PRDC)*, Dec. 2019, p. 256.
- [6] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *J. Inf. Secur. Appl.*, vol. 50, Feb. 2020, Art. no. 102419.
- [7] T. Hasan, J. Malik, I. Bibi, W. U. Khan, F. N. Al-Wesabi, K. Dev, and G. Huang, "Securing industrial Internet of Things against botnet attacks using hybrid deep learning approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2952–2963, Sep./Oct. 2023.
- [8] D. T. Son, N. T. K. Tram, and P. M. Hieu, "Deep learning techniques to detect botnet," *J. Sci. Technol. Inf. Secur.*, vol. 1, no. 15, pp. 85–91, Jun. 2022.
- [9] M. Gandhi and S. Srivatsa, "Detecting and preventing attacks using network intrusion detection systems," *Int. J. Comput. Sci. Secur.*, vol. 2, no. 1, pp. 49–60, 2008.
- [10] J. Liu, S. Liu, and S. Zhang, "Detection of IoT botnet based on deep learning," in *Proc. Chin. Control Conf. (CCC)*, 2019, pp. 8381–8385.
- [11] C. D. McDermott, F. Majdani, and A. V. Petrovski, "Botnet detection in the Internet of Things using deep learning approaches," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2018, pp. 1–8.
- [12] S. Sriram, R. Vinayakumar, M. Alazab, and K. Soman, "Network flow based IoT botnet attack detection using deep learning," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*

Workshops (INFOCOM WKSHPS), Jul. 2020, pp. 189–194.

[13] B. Nugraha, A. Nambiar, and T. Bauschert, "Performance evaluation of botnet detection using deep learning techniques," in *Proc. 11th Int. Conf. Netw. Future (NoF)*, Oct. 2020, pp. 141–149.

[14] P. Karunakaran, "Deep learning approach to DGA classification for effective cyber security," *J. Ubiquitous Comput. Commun. Technol. (UCCT)*, vol. 2, no. 4, pp. 203–213, 2020.

[15] N. Elsayed, Z. ElSayed, and M. Bayoumi, "IoT botnet detection using an economic deep learning model," *arXiv:2302.02013*, 2023.

[16] M. A. Haq and M. A. Rahim Khan, "DNNBoT: Deep neural network based botnet detection and classification," *Comput. Mater. Continua*, vol. 71, no. 1, pp. 1729–1750, 2022.

[17] I. H. Sarker, "Deep cybersecurity: A comprehensive overview from neural network and deep learning perspective," *SN Comput. Sci.*, vol. 2, no. 3, p. 154, May 2021.

[18] A. A. Ahmed, W. A. Jabbar, A. S. Sadiq, and H. Patel, "Deep learning based classification model for botnet attack detection," *J. Ambient Intell. Humanized Comput.*, vol. 13, no. 7, pp. 3457–3466, Jul. 2022.

[19] I. Letteri, M. Del Rosso, P. Caianiello, and D. Cassioli, "Performance of botnet detection by neural networks in software-defined networks," in *Proc. ITASEC*, 2018, pp. 1–10.

[20] T. H. H. Aldhyani and H. Alkahtani, "Attacks to automotous vehicles: A deep learning algorithm for cybersecurity," *Sensors*, vol. 22, no. 1, p. 360, Jan. 2022.

[21] M. Y. Alzahrani and A. M. Bamhdi, "Hybrid deep-learning model to detect botnet attacks over Internet of Things environments," *Soft Comput.*, vol. 26, no. 16, pp. 7721–7735, Aug. 2022.

[22] Y. N. Soe, P. I. Santosa, and R. Hartanto, "DDoS attack detection based on simple ANN with SMOTE for IoT environment," in *Proc. 4th Int. Conf. Informat. Comput. (ICIC)*, Oct. 2019, pp. 1–5.

[23] S.-C. Chen, Y.-R. Chen, and W.-G. Tzeng, "Effective botnet detection through neural networks on convolutional features," in *Proc. 17th IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun./12th IEEE Int. Conf. Big Data Sci. Eng. (TrustCom/BigDataSE)*, Aug. 2018, pp. 372–378.

[24] S. I. Popoola, B. Adebisi, M. Hammoudeh, G. Gui, and H. Gacanin, "Hybrid deep learning for botnet attack detection in the Internet of Things networks," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4944–4956, Mar. 2021.

[25] S. Akarsh, S. Sriram, P. Poornachandran, V. K. Menon, and K. P. Soman, "Deep

learning framework for domain generation algorithms prediction using long short-term memory," in *Proc. 5th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS)*, Mar. 2019, pp. 666–671.

[26] S. I. Popoola, B. Adebisi, M. Hammoudeh, H. Gacanin, and G. Gui, "Stacked recurrent neural network for botnet detection in smart homes," *Comput. Electr. Eng.*, vol. 92, Jun. 2021, Art. no. 107039.

[27] M. Alauthman, "Botnet spam e-mail detection using deep recurrent neural network," *Int. J. Emerg. Trends Eng. Res.*, vol. 8, no. 5, pp. 1979–1986, May 2020.

[28] J. Bhayo, S. A. Shah, S. Hameed, A. Ahmed, J. Nasir, and D. Draheim, "Towards a machine learning-based framework for DDoS attack detection in software-defined IoT (SD-IoT) networks," *Eng. Appl. Artif. Intell.*, vol. 123, Aug. 2023, Art. no. 106432.

[29] S. Siddiqui, S. Hameed, S. A. Shah, I. Ahmad, A. Aneiba, D. Draheim, and S. Dustdar, "Toward software-defined networking-based IoT frameworks: A systematic literature review, taxonomy, open challenges and prospects," *IEEE Access*, vol. 10, pp. 70850–70901, 2022.

[30] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Gener. Comput. Syst.*, vol. 100, pp. 779–796, Nov. 2019.

[31] M. Khalid, S. Hameed, A. Qadir, S. A. Shah, and D. Draheim, "Towards SDN-based smart contract solution for IoT access control," *Comput. Commun.*, vol. 198, pp. 1–31, Jan. 2023.